

Deep Learning

**A Napkin-style companion for campus placements:
from the perceptron to Transformers and LLMs, with derivations,
hand numerics, traps, and code that checks itself**

Philospher

Built October 6, 2026

Preface

[STUB: preface] Versions: 2.14.1+cu130.

Contents

Preface	iii
I Foundations	7
1 Perceptron and the multilayer perceptron	9
1.1 The artificial neuron	9
1.2 The perceptron	10
1.3 XOR: what one neuron cannot do	12
1.4 The multilayer perceptron	14
1.5 Universal approximation, and why depth still matters	15
1.6 Capacity: what makes a network more expressive	17
1.7 A few problems to think about	18
2 Activations, losses and output layers	19
2.1 Activation functions	19
2.2 Output layers: sigmoid and softmax	21
2.3 Loss functions	23
2.4 A few problems to think about	26
3 Backpropagation	27
3.1 Computation graphs and the chain rule	27
3.2 Backpropagation for an MLP, in matrix form	28
3.3 A complete forward and backward pass by hand	29
3.4 Gradient checking	30
3.5 Reverse-mode autograd from scratch	31
3.6 Vanishing and exploding gradients	33
3.7 What backprop costs	34
3.8 A few problems to think about	34
4 Optimisers and learning-rate schedules	37
4.1 Gradient descent and its one big problem	37
4.2 Momentum and Nesterov	38
4.3 Adaptive methods: AdaGrad, RMSProp, Adam	38
4.4 Paths and curves	41
4.5 Learning-rate schedules	42
4.6 Batch size, clipping and the loss landscape	43
4.7 A few problems to think about	44
5 Initialisation, normalisation and regularisation	45
5.1 Initialisation, normalisation and regularisation	45
II Architectures	47
6 Convolutional networks	49
6.1 Convolutional networks	49

7	Recurrent networks	51
7.1	Recurrent networks	51
8	Attention and the Transformer	53
8.1	Attention and the Transformer	53
III	Modern Deep Learning	55
9	Language models and LLMs	57
9.1	Language models and LLMs	57
10	Generative and representation learning	59
10.1	Generative and representation learning	59
IV	Interview Mode	61
11	Practice and PyTorch	63
11.1	Practice and PyTorch	63
12	Implement it	65
12.1	Implement it	65
13	Rapid-fire	67
13.1	Rapid-fire	67
14	Problems	69
14.1	Problems	69
V	Backmatter	71
A	Hints to problems	73
B	Solutions to problems	75
	Bibliography	77

I

Foundations

Part I: Contents

1	Perceptron and the multilayer perceptron	9
1.1	The artificial neuron	9
1.2	The perceptron	10
1.3	XOR: what one neuron cannot do	12
1.4	The multilayer perceptron	14
1.5	Universal approximation, and why depth still matters	15
1.6	Capacity: what makes a network more expressive	17
1.7	A few problems to think about	18
2	Activations, losses and output layers	19
2.1	Activation functions	19
2.2	Output layers: sigmoid and softmax	21
2.3	Loss functions	23
2.4	A few problems to think about	26
3	Backpropagation	27
3.1	Computation graphs and the chain rule	27
3.2	Backpropagation for an MLP, in matrix form	28
3.3	A complete forward and backward pass by hand	29
3.4	Gradient checking	30
3.5	Reverse-mode autograd from scratch	31
3.6	Vanishing and exploding gradients	33
3.7	What backprop costs	34
3.8	A few problems to think about	34
4	Optimisers and learning-rate schedules	37
4.1	Gradient descent and its one big problem	37
4.2	Momentum and Nesterov	38
4.3	Adaptive methods: AdaGrad, RMSProp, Adam	38
4.4	Paths and curves	41
4.5	Learning-rate schedules	42
4.6	Batch size, clipping and the loss landscape	43
4.7	A few problems to think about	44
5	Initialisation, normalisation and regularisation	45
5.1	Initialisation, normalisation and regularisation	45

1 Perceptron and the multilayer perceptron

Every network in this book, from a two-layer classifier to a Transformer with a hundred layers, is built from one object: a weighted sum followed by a fixed non-linear function. This chapter takes that object apart, shows exactly what one of them can and cannot do (the XOR story that OAs keep asking about), and then stacks them into the multilayer perceptron whose parameters we learn to count in our sleep.

§1.1 The artificial neuron

Prototypical example for this section: Inputs $\mathbf{x} = (0.5, -1, 2)$, weights $\mathbf{w} = (0.4, 0.3, -0.2)$, bias $b = 0.1$: the neuron outputs $\phi(\mathbf{w}^\top \mathbf{x} + b)$ for some transfer function ϕ .

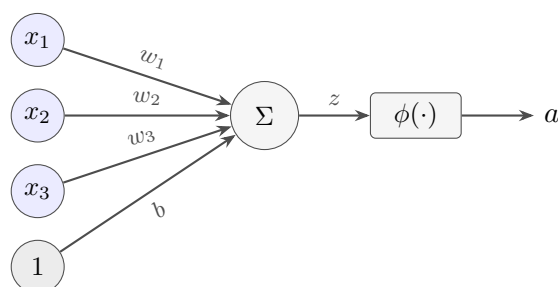
§1.1.i Definition

Definition 1.1.1. An **artificial neuron** with weights $\mathbf{w} \in \mathbb{R}^d$, bias $b \in \mathbb{R}$ and **activation** (or **transfer**) function $\phi : \mathbb{R} \rightarrow \mathbb{R}$ maps an input $\mathbf{x} \in \mathbb{R}^d$ to

$$a = \phi(z), \quad z = \mathbf{w}^\top \mathbf{x} + b = \sum_{i=1}^d w_i x_i + b.$$

The number z is the **pre-activation** (or weighted sum, or logit); a is the **activation**.

The classical choices of ϕ are the **step** (Heaviside) function $\mathbb{K}[z \geq 0]$, the **sigmoid** $\sigma(z) = 1/(1 + e^{-z})$, $\tanh z$ and the **ReLU** $\max(0, z)$. **Chapter 2** compares them properly; for now they are just four ways to squash or clip z .



Example 1.1.2 (One neuron, four transfer functions)

With the prototype numbers, $z = 0.4(0.5) + 0.3(-1) + (-0.2)(2) + 0.1 = -0.40$. Then

ϕ	step	sigmoid	tanh	ReLU
$a = \phi(z)$	0.0000	0.4013	-0.3799	0.0000

All four agree with `torch.sigmoid`, `torch.tanh` and `torch.relu` (checked in `code/dl/c01_perceptron.py`). OA version: inputs $(1, 0, 1, 1)$, weights $(0.6, -0.9, -0.4, 0.5)$, bias -0.3 , sigmoid transfer. Then $z = 0.40$ and the output is $\sigma(z) = 0.5987$. The input that is 0 contributes nothing whatever its weight,

which is exactly what the setter hopes you will forget.

Remark 1.1.3 — Trap. The neuron applies ϕ to the weighted sum; it does not take a weighted sum of $\phi(x_i)$. A second common slip is dropping the bias: then every decision boundary is forced through the origin.

§1.1.ii The biological analogy, and where it breaks

The name comes from a cartoon of a cortical neuron: dendrites collect signals from other cells, synapses scale them (the weights), the cell body integrates them (the sum), and the axon fires if the total crosses a threshold (the step function). The cartoon is useful for exactly one sentence. It breaks almost everywhere an interviewer might push:

- Real neurons communicate with spikes in time; information sits in rates and timing, not in one real number per forward pass.
- Dendrites themselves compute non-linear functions, so one biological neuron is closer to a small network than to one unit.
- Nobody has shown that the brain runs backpropagation: it would need the backward pass to reuse the forward weights exactly (“weight transport”).
- An artificial neuron with a sigmoid is literally logistic regression (see the logistic-regression chapter of the companion book *Classical Machine Learning*); the interesting behaviour comes from composing many of them and training them jointly.

A neuron is a linear function followed by a fixed scalar non-linearity. Everything else is composition.

§1.2 The perceptron

Prototypical example for this section: Six labelled points in the plane; Rosenblatt’s rule adds misclassified points to $\mathbf{w}$ until every point is on the right side.

§1.2.i The algorithm

Rosenblatt’s perceptron [R58] is a step-function neuron $\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$ with labels $y \in \{-1, +1\}$ and a learning rule that only reacts to mistakes.

Algorithm: Perceptron

Initialise $\mathbf{w} = \mathbf{0}$, $b = 0$. Repeat over the data until an epoch makes no mistake: if $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \leq 0$ then $\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \mathbf{x}_i$ and $b \leftarrow b + \eta y_i$.

Why it is the right direction: after the update the new margin of the same point is $y_i(\mathbf{w} + \eta y_i \mathbf{x}_i)^\top \mathbf{x}_i + y_i(b + \eta y_i) = y_i(\mathbf{w}^\top \mathbf{x}_i + b) + \eta(\|\mathbf{x}_i\|^2 + 1)$, strictly larger than before. The rule is stochastic (sub)gradient descent on the **perceptron loss** $\ell(\mathbf{w}, b) = \max(0, -y(\mathbf{w}^\top \mathbf{x} + b))$, whose gradient is $-y\mathbf{x}$ on a mistake and 0 otherwise.

code/dl/c01_perceptron.py

```

1 def perceptron(X, y, epochs=20, lr=1.0):
2     w, b = np.zeros(X.shape[1]), 0.0
3     log = []
4     for ep in range(epochs):
5         mistakes = 0
6         for i in range(len(X)):
7             if y[i] * (w @ X[i] + b) <= 0:      # mistake (or on the boundary)
8                 w, b = w + lr * y[i] * X[i], b + lr * y[i]
9                 mistakes += 1
10                log.append((ep + 1, i + 1, w.copy(), b))
11        if mistakes == 0:
12            return w, b, log, ep + 1
13    return w, b, log, epochs

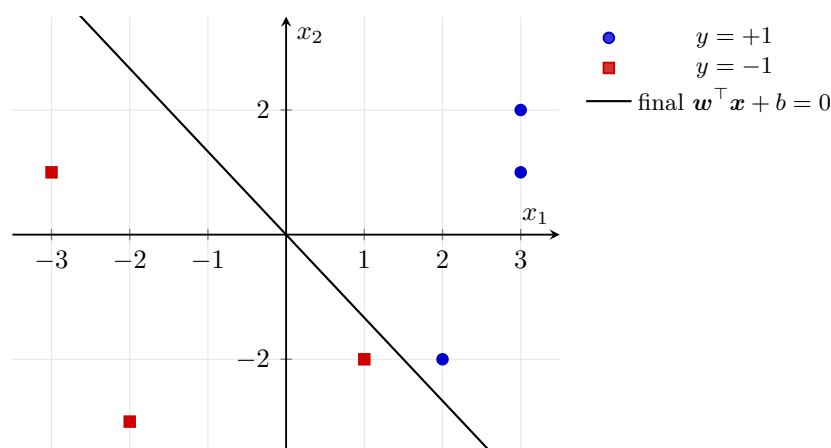
```

Example 1.2.1 (A full trace)

Data: $\mathbf{x}_1 = (3, 1)^+$, $\mathbf{x}_2 = (1, -2)^-$, $\mathbf{x}_3 = (3, 2)^+$, $\mathbf{x}_4 = (-2, -3)^-$, $\mathbf{x}_5 = (2, -2)^+$, $\mathbf{x}_6 = (-3, 1)^-$; $\eta = 1$, points visited in index order. Every update made:

epoch	mistake on	$\mathbf{w}$ after	b after
1	$\mathbf{x}_1$	(3, 1)	1
1	$\mathbf{x}_2$	(2, 3)	0
1	$\mathbf{x}_5$	(4, 1)	1
2	$\mathbf{x}_2$	(3, 3)	0
2	$\mathbf{x}_5$	(5, 1)	1
3	$\mathbf{x}_2$	(4, 3)	0

Epoch 4 makes no mistake, so the algorithm stops after 6 updates at $\mathbf{w} = (4, 3)$, $b = 0$. The result is identical to `sklearn.linear_model.Perceptron(eta0=1, shuffle=False)`, which the script asserts. Notice $\mathbf{x}_2$ and $\mathbf{x}_5$, the two points closest to each other with opposite labels, cause every update after the first.



Remark 1.2.2 — Trap. “Increase the learning rate to make the perceptron converge faster.” With $\mathbf{w}_0 = \mathbf{0}$, every weight vector the algorithm visits is η times the one it visits with $\eta = 1$, so the signs of all predictions, the mistakes and the number of updates are identical. The script reruns the trace with $\eta = 0.1$ and asserts the final

weights are exactly 0.1 times the ones above. The learning rate of the perceptron only matters when $\mathbf{w}_0 \neq \mathbf{0}$.

§1.2.ii The convergence theorem

Theorem 1.2.3 (Perceptron convergence)

Suppose $\|(\mathbf{x}_i, 1)\| \leq R$ for all i and some unit vector $\mathbf{w}^*$ (acting on augmented inputs $(\mathbf{x}, 1)$) satisfies $y_i \mathbf{w}^{*\top}(\mathbf{x}_i, 1) \geq \gamma > 0$ for all i . Then the perceptron makes at most $(R/\gamma)^2$ updates, whatever the order of the data.

Proof. Absorb b into $\mathbf{w}$ by appending 1 to every input, and take $\eta = 1$ (by the remark above this costs nothing). Let $\mathbf{w}_k$ be the weights after k updates, the k -th made on example i . Two inequalities:

- Progress: $\mathbf{w}_k^\top \mathbf{w}^* = \mathbf{w}_{k-1}^\top \mathbf{w}^* + y_i \mathbf{x}_i^\top \mathbf{w}^* \geq \mathbf{w}_{k-1}^\top \mathbf{w}^* + \gamma$, so $\mathbf{w}_k^\top \mathbf{w}^* \geq k\gamma$.
- Bounded growth: $\|\mathbf{w}_k\|^2 = \|\mathbf{w}_{k-1}\|^2 + 2y_i \mathbf{w}_{k-1}^\top \mathbf{x}_i + \|\mathbf{x}_i\|^2 \leq \|\mathbf{w}_{k-1}\|^2 + R^2$ because the update happened only when $y_i \mathbf{w}_{k-1}^\top \mathbf{x}_i \leq 0$. So $\|\mathbf{w}_k\|^2 \leq kR^2$.

By Cauchy–Schwarz $k\gamma \leq \mathbf{w}_k^\top \mathbf{w}^* \leq \|\mathbf{w}_k\| \leq \sqrt{k}R$, hence $k \leq (R/\gamma)^2$. $\square$

Example 1.2.4 (How loose is the bound?)

For the six points above, $R = 3.742$ and the best possible margin (from a hard-margin linear SVM, normalised) is $\gamma = 0.277$, so the bound is $(R/\gamma)^2 \approx 182.0$ updates. The perceptron needed 6. The bound is a worst case over orderings and is driven by the single tightest pair of points.

Question 1.2.5. Why does the rule update when $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \leq 0$ rather than < 0 ? (Start from $\mathbf{w} = \mathbf{0}$.)

Remark 1.2.6 — Trap. The theorem says nothing about non-separable data, where the perceptron never stops, and nothing about *which* separator it returns: unlike an SVM it does not maximise the margin, and the answer depends on the order of the data.

§1.3 XOR: what one neuron cannot do

Prototypical example for this section: $\text{XOR}(x_1, x_2)$ is 1 exactly when one input is 1; no line separates $\{(0, 1), (1, 0)\}$ from $\{(0, 0), (1, 1)\}$.

Proposition 1.3.1

No single step-function neuron computes XOR.

Proof. Suppose $\mathbb{K}[w_1x_1 + w_2x_2 + b \geq 0]$ equals XOR. The four inputs give $b < 0$, $w_1 + w_2 + b < 0$, $w_2 + b \geq 0$ and $w_1 + b \geq 0$. Adding the last two, $w_1 + w_2 + 2b \geq 0$, so $w_1 + w_2 + b \geq -b > 0$, contradicting the second. $\square$

The same argument works for any monotone ϕ thresholded at any level, so “use a sigmoid instead” does not help: a single neuron’s decision boundary is always a hyperplane. This observation, made precise by Minsky and Papert [MiPa69], is the usual one-line answer to “why do we need hidden layers?”.

Example 1.3.2 (The perceptron on XOR)

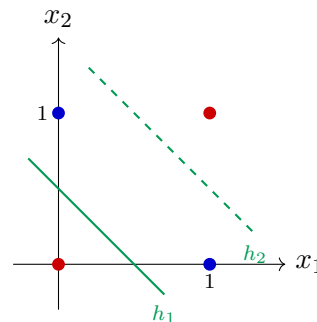
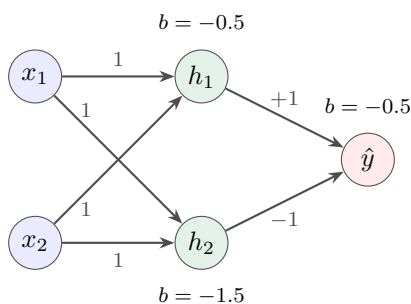
Running the perceptron for 50 epochs on XOR (labels ± 1) gives 200 updates in total and never fewer than 4 mistakes in an epoch: the weights cycle forever. `sklearn`’s perceptron ends at training accuracy 0.50, no better than a coin.

§1.3.i Two layers solve it by hand

XOR = OR and not AND. Each of OR and AND is linearly separable, and so is “ h_1 and not h_2 ” once h_1, h_2 are available as features:

code/dl/c01_perceptron.py

```
1 step = lambda t: (t >= 0).astype(float)
2 def xor_net(X):
3     h1 = step(X @ np.array([1, 1]) - 0.5)    # OR
4     h2 = step(X @ np.array([1, 1]) - 1.5)    # AND
5     return step(h1 - h2 - 0.5), h1, h2      # OR and not AND
```



x_1	x_2	$h_1 = \text{OR}$	$h_2 = \text{AND}$	$\hat{y}$
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	0

The hidden layer *re-represents* the input: in (h_1, h_2) coordinates the four points become $(0,0)$, $(1,0)$, $(1,0)$, $(1,1)$ and the two classes are linearly separable. That is the whole idea of representation learning in one table.

Example 1.3.3 (Can gradient descent find it?)

Train a 2- H -1 network with tanh hidden units and a sigmoid output on the four XOR points with Adam, from 20 random initialisations. With $H = 2$ (the minimum that can represent XOR) 10 of the 20 runs reach 100% accuracy; the rest stall in a poor local configuration. With $H = 8$, 20 of 20 succeed. Over-parameterising makes the optimisation problem easier, a theme that returns with every large model

in this book.

§1.4 The multilayer perceptron

Prototypical example for this section: A 784-256-128-10 ReLU network for MNIST digits has 235,146 parameters.

§1.4.i Forward pass and shapes

A **multilayer perceptron** (MLP, also **fully connected** or **dense** network) with layer widths $n_0, n_1, \dots, n_L$ computes

$$\mathbf{h}^{(0)} = \mathbf{x}, \quad \mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{h}^{(l)} = \phi(\mathbf{z}^{(l)}) \quad (l = 1, \dots, L),$$

with $\mathbf{W}^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ and $\mathbf{b}^{(l)} \in \mathbb{R}^{n_l}$. The last layer usually has no ϕ (it outputs logits; [chapter 2](#)). With a batch $\mathbf{X} \in \mathbb{R}^{B \times n_0}$ stored row-wise, as every framework does, the same layer reads $\mathbf{Z} = \mathbf{XW}^\top + \mathbf{1b}^\top$, shape (B, n_l) ; PyTorch's `nn.Linear(n_in, n_out)` stores `weight` with shape (n_out, n_in) for exactly this reason.

Proposition 1.4.1 (Parameter count)

An MLP with widths $n_0, \dots, n_L$ has $\sum_{l=1}^L (n_{l-1} + 1) n_l$ parameters: each of the n_l units in layer l has n_{l-1} weights and one bias.

Example 1.4.2 (Counting an MNIST MLP)

For $784 \rightarrow 256 \rightarrow 128 \rightarrow 10$: $785 \cdot 256 = 200,960$, $257 \cdot 128 = 32,896$, $129 \cdot 10 = 1,290$; total 235,146, equal to `sum(p.numel() for p in model.parameters())` for the PyTorch model. A 3-4-2 network has 26 parameters, a typical one-line OA item. Activation functions and the input layer have no parameters.

code/dl/c01_perceptron.py

```
1 def mlp_params(sizes):
2     return sum(m * n + n for m, n in zip(sizes[:-1], sizes[1:]))
3 sizes = [784, 256, 128, 10]
4 n_ours = mlp_params(sizes)
```

§1.4.ii Why the non-linearity is not optional

Delete every ϕ and the network collapses: $\mathbf{W}^{(2)}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)} = (\mathbf{W}^{(2)}\mathbf{W}^{(1)})\mathbf{x} + (\mathbf{W}^{(2)}\mathbf{b}^{(1)} + \mathbf{b}^{(2)})$, a single affine map, however many layers you stack (the script checks this numerically). A deep linear network can represent only what one linear layer can, so it cannot solve XOR either.

Remark 1.4.3 — Trap. “ReLU is linear, so it adds no non-linearity.” ReLU is *piecewise* linear: it is linear on each side of 0 but not linear overall ($\text{ReLU}(-1) + \text{ReLU}(1) \neq \text{ReLU}(0)$). A ReLU network is a piecewise-linear function with possibly a huge number of pieces, which is exactly what the next section exploits.

§1.5 Universal approximation, and why depth still matters

Prototypical example for this section: Thirty-two ReLU units in one hidden layer approximate $\sin(2\pi t) + t/2$ on $[0, 1]$ to within 0.0048.

Theorem 1.5.1 (Universal approximation, informal)

Let ϕ be continuous and not a polynomial. For every continuous $f : K \rightarrow \mathbb{R}$ on a compact $K \subset \mathbb{R}^d$ and every $\varepsilon > 0$ there is a network with one hidden layer of ϕ units and a linear output whose output is within ε of f everywhere on K .

Cybenko proved it for sigmoids [Cy89]; later work extended it to any continuous non-polynomial activation. Two caveats make up the interview answer: the theorem says a suitable width *exists* but not how large it is (it can be exponential in d), and it says nothing about whether gradient descent will find the weights or whether they generalise.

§1.5.i A constructive proof in one dimension

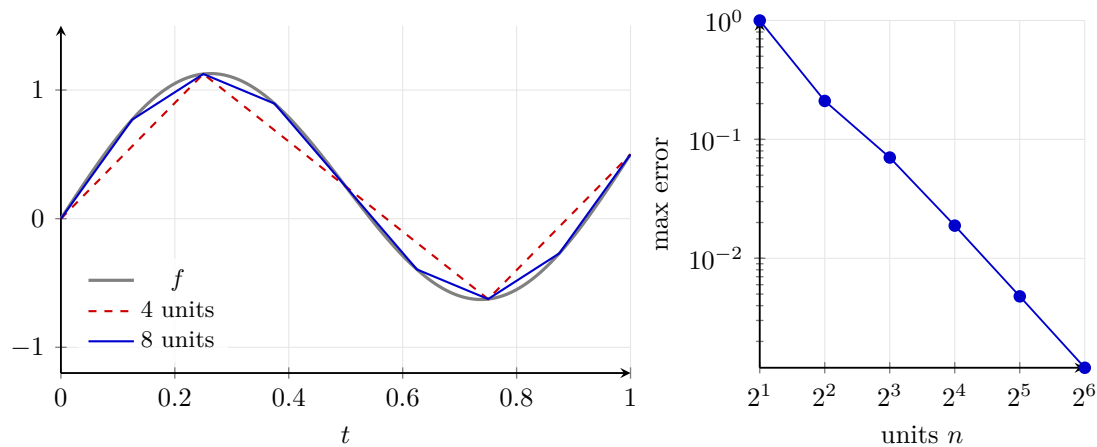
With ReLUs the 1-D case is a construction you can write in an interview. Take knots $0 = t_0 < t_1 < \dots < t_n = 1$ and the piecewise-linear interpolant of f . Its slope on $[t_k, t_{k+1}]$ is s_k . Then

$$\hat{f}(t) = f(0) + \sum_{k=0}^{n-1} a_k \operatorname{ReLU}(t - t_k), \quad a_0 = s_0, \quad a_k = s_k - s_{k-1},$$

because the unit $\operatorname{ReLU}(t - t_k)$ switches on at t_k and changes the slope by a_k there. That is a one-hidden-layer network with n units: hidden weights 1, hidden biases $-t_k$, output weights a_k .

code/dl/c01_perceptron.py

```
1 def relu_net_interp(f, n):
2     """One hidden ReLU layer with n units that interpolates f at n+1 knots."""
3     knots = np.linspace(0, 1, n + 1)
4     vals = f(knots)
5     slopes = np.diff(vals) / np.diff(knots)
6     a = np.r_[slopes[0], np.diff(slopes)] # output weights
7     def net(t):
8         H = np.maximum(0.0, t[:, None] - knots[:-1][None, :]) # hidden layer
9         return vals[0] + H @ a
10    return net
```



Example 1.5.2 (Error rate of the construction)

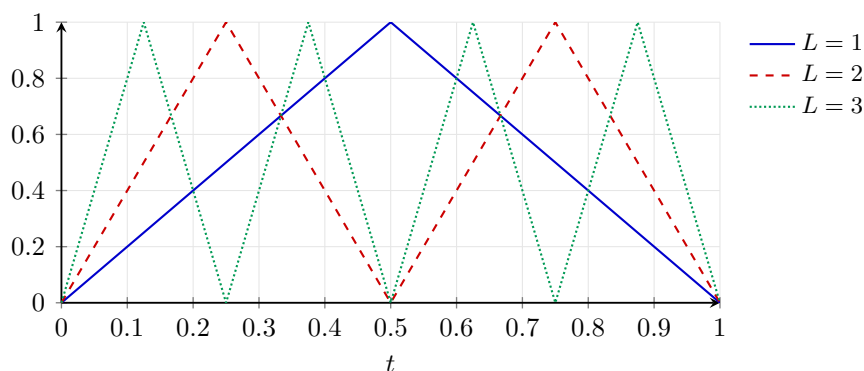
The script verifies that the network equals `numpy.interp` at 401 points and measures the worst error: 0.2105, 0.0704, 0.0188, 0.0048, 0.0012 for $n = 4, 8, 16, 32, 64$. Doubling n divides the error by about 3.98, matching the interpolation bound $h^2 \max |f''|/8$ with $h = 1/n$; for $n = 64$ that bound is 0.0012. In d dimensions the same grid argument needs n^d cells: the curse of dimensionality hiding inside “universal”.

§1.5.ii Depth buys pieces exponentially

A one-hidden-layer ReLU net on $\mathbb{R}$ with m units is piecewise linear with at most $m + 1$ pieces (each unit adds at most one kink). Now compose the tent map $g(t) = 2 \operatorname{ReLU}(t) - 4 \operatorname{ReLU}(t - \frac{1}{2})$, which uses 2 ReLU units and maps $[0, 1]$ onto $[0, 1]$ up and back down. Composing it L times gives a sawtooth.

code/dl/c01_perceptron.py

```
1 def tent_layer(t):          # 2 ReLU units: g(t) = 2 relu(t) - 4 relu(t - 1/2)
2     return 2 * np.maximum(0, t) - 4 * np.maximum(0, t - 0.5)
3 def deep_tent(t, L):
4     for _ in range(L):
5         t = tent_layer(t)
6     return t
```



Counting slope changes numerically, depth $L = 1, \dots, 6$ gives 2, 4, 8, 16, 32, 64 linear pieces, i.e. 2^L , using only $2L$ units. The depth-6 sawtooth uses 12 units; a single hidden layer needs at least 63 to produce 64 pieces. Depth multiplies, width adds.

One hidden layer is enough in principle; depth is what makes it enough in practice.

§1.6 Capacity: what makes a network more expressive

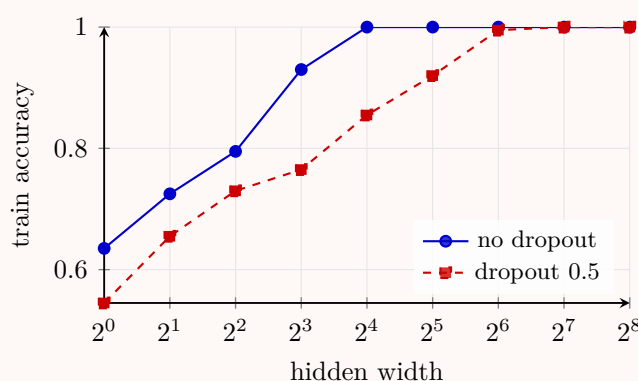
Prototypical example for this section: “Which of these increases a network’s capacity: more layers, dropout, a larger learning rate, more data?” Only the first.

Capacity is the richness of the set of functions the architecture can represent (its hypothesis class). It is a property of the model, not of the data or of the optimiser.

- **Increase capacity:** more layers, more units per layer, more input features, a richer activation, fewer constraints (e.g. removing weight sharing).
- **Reduce effective capacity:** dropout, weight decay, early stopping, data augmentation, weight sharing, bottleneck layers. These restrict which functions training can reach or prefers.
- **Do not change capacity:** the learning rate, the batch size, the number of epochs as such, and the amount of data. They change what training *finds*, not what the model *can* represent.

Example 1.6.1 (Memorising random labels)

Give 200 random points in $\mathbb{R}^{10}$ coin-flip labels, so the only way to fit them is to memorise. Train one-hidden-layer ReLU nets of increasing width to convergence. Training accuracy rises from 63.5% at width 1 to 100.0% at width 16. With dropout $p = 0.5$ the width-16 net reaches only 85.5%: the same architecture, lower effective capacity.



Remark 1.6.2 — Trap. “Dropout increases capacity because it trains an ensemble.” Dropout trains an implicit ensemble of thinned sub-networks that share weights; each sub-network is smaller than the full one, and the averaged predictor is pulled toward simpler functions. It is a regulariser: it lowers effective capacity (chapter 5).

Remark 1.6.3 — Trap. “A bigger learning rate lets the model fit more complex functions.” It changes the path of optimisation, sometimes for the better (noise that escapes sharp minima), but the set of representable functions is fixed by the

architecture.

§1.7 A few problems to think about

Problem 1A. How many trainable parameters does a fully connected network with layer sizes $100 \rightarrow 50 \rightarrow 50 \rightarrow 3$ have (biases included)?

[hint · solution](#)

Problem 1B. Give weights and a bias for a single step neuron computing NAND on $\{0, 1\}^2$. Why does this show that networks of step neurons can compute any Boolean function?

[hint · solution](#)

Problem 1C. A perceptron *without bias* is trained from $\mathbf{w} = \mathbf{0}$ on linearly separable data. Every input is then multiplied by $c > 0$ and training is rerun. How do R , γ , the bound $(R/\gamma)^2$ and the sequence of mistakes change?

[hint · solution](#)

Problem 1D. Show that the one-hidden-layer ReLU network on $\mathbb{R}$

$$f(t) = c + \sum_{k=1}^m a_k \text{ReLU}(w_k t + b_k)$$

is piecewise linear with at most $m + 1$ pieces.

[hint · solution](#)

Problem 1E. Which statements are true? (a) A single sigmoid neuron can represent XOR. (b) Stacking linear layers without activations increases representational power. (c) Adding hidden layers increases capacity. (d) Dropout increases capacity. (e) A one-hidden-layer network can approximate any continuous function on a compact set, given enough units.

[hint · solution](#)

Problem 1F. Prove that the perceptron update never decreases $\mathbf{w}^\top \mathbf{w}^*$ and explain in one sentence why that alone does not prove convergence.

[hint · solution](#)

2 Activations, losses and output layers

Three choices sit at fixed places in every network: the non-linearity between layers, the function that turns the last layer’s numbers into a prediction, and the loss that scores that prediction. OAs test all three with one-minute numerics (“compute the softmax of these five logits”) and with classification questions (“which of these is a loss and which is an activation?”). Interviews test whether you know why the standard pairings are standard.

§2.1 Activation functions

Prototypical example for this section: Sigmoid, tanh, ReLU and GELU evaluated at $z = -2$ give four different answers, and the derivative of the sigmoid is never larger than 0.25.

§2.1.i What the activation is for

Chapter 1 showed that without an activation a deep network collapses to one affine map. The activation is the *only* source of non-linearity in an MLP: weights and biases enter linearly, and the loss is applied after the network, so it cannot change what functions the network represents.

Remark 2.1.1 — Trap. “Which component provides non-linearity: weights, biases, activation function, loss function?” Only the activation function. A non-linear *loss* (cross-entropy is very non-linear) does not help: a linear model trained with cross-entropy is still logistic regression, with a linear boundary.

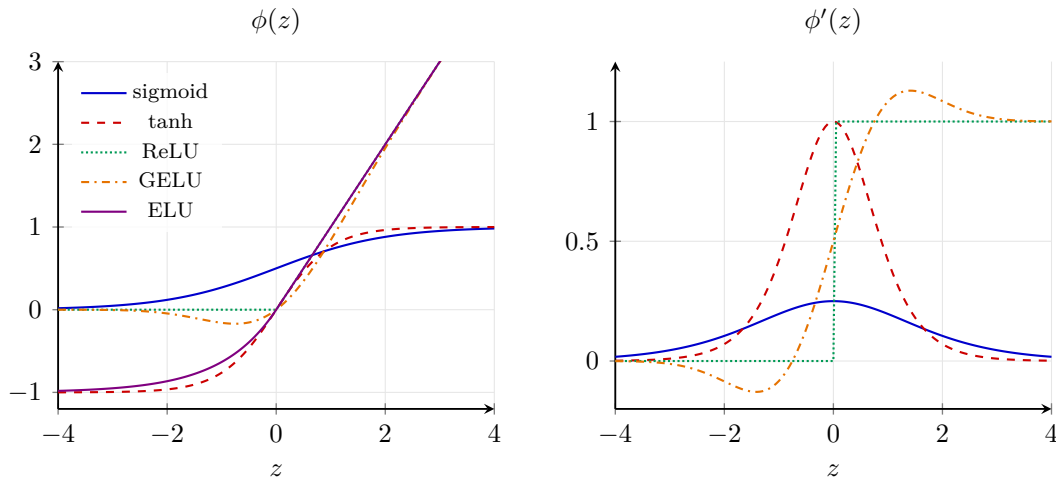
§2.1.ii The zoo, with values

name	$\phi(z)$	$\phi'(z)$
sigmoid	$\sigma(z) = 1/(1 + e^{-z})$	$\sigma(z)(1 - \sigma(z)) \leq \frac{1}{4}$
tanh	$(e^z - e^{-z})/(e^z + e^{-z}) = 2\sigma(2z) - 1$	$1 - \tanh^2 z \leq 1$
ReLU	$\max(0, z)$	$\mathbb{K}[z > 0]$
leaky ReLU	$\max(\alpha z, z), \alpha \approx 0.01$	1 or α
ELU	z if $z > 0$, else $\alpha(e^z - 1)$	1 or αe^z
GELU	$z \Phi(z)$, Φ the standard normal CDF	$\Phi(z) + z \varphi(z)$
SiLU / swish	$z \sigma(z)$	$\sigma(z)(1 + z(1 - \sigma(z)))$
softplus	$\log(1 + e^z)$	$\sigma(z)$

Example 2.1.2 (Values an OA might ask for)

Computed by `code/dl/c02_activations.py`, each function checked against `torch.nn.functional` on a grid of 161 points and each derivative against autograd:

ϕ	$z = -2$	-0.5	0	0.5	2
sigmoid	0.1192	0.3775	0.5000	0.6225	0.8808
tanh	-0.9640	-0.4621	0.0000	0.4621	0.9640
ReLU	0.0000	0.0000	0.0000	0.5000	2.0000
leaky ReLU (0.01)	-0.0200	-0.0050	0.0000	0.5000	2.0000
ELU ($\alpha = 1$)	-0.8647	-0.3935	0.0000	0.5000	2.0000
GELU	-0.0455	-0.1543	0.0000	0.3457	1.9545
SiLU	-0.2384	-0.1888	0.0000	0.3112	1.7616



§2.1.iii Sigmoid and tanh: saturation

Both squash $\mathbb{R}$ into a bounded interval, and both are nearly flat away from 0: $\sigma'(5) = 0.0066$. Backpropagation multiplies one such factor per layer (chapter 3), and since $\sigma' \leq \frac{1}{4}$ ten sigmoid layers shrink a gradient by at least 4^{10} , i.e. to at most 9.54×10^{-7} of its size from that factor alone. This is the **vanishing gradient** problem in its simplest form.

The sigmoid has a second defect: its outputs are all positive. For a neuron fed by sigmoid units, $\partial \mathcal{L} / \partial w_i = \delta \cdot h_i$ with every $h_i > 0$, so all of that neuron's weight gradients share the sign of δ and the weights can only move together up or together down, giving zig-zag updates. tanh is zero-centred and its maximum slope is 1, so it was preferred for hidden layers even before ReLU. Today sigmoid is used at outputs (probabilities) and in gates (LSTM, chapter 7), almost never in hidden layers.

§2.1.iv ReLU and its relatives

ReLU does not saturate for $z > 0$ (derivative exactly 1), is cheap, and produces exact zeros (sparse activations). Its failure mode is the **dead ReLU**: a unit whose pre-activation is negative on every training input gets zero gradient forever (chapter 3 has a simulation). Leaky ReLU and PReLU (learned α) keep a small slope for $z < 0$; ELU makes the negative side smooth and pushes mean activations toward zero.

Remark 2.1.3 — Trap. “ReLU is not differentiable at 0, so it cannot be trained by gradient descent.” The kink is a single point that floating-point inputs essentially never hit exactly; frameworks use the subgradient 0 there, and ReLU is the default

hidden activation of most CNNs.

§2.1.v Smooth modern activations: GELU and SiLU

GELU weights the input by the probability that a standard normal falls below it, $z \Phi(z)$; Transformers (BERT, GPT-style models) use it or SiLU in their feed-forward blocks [HeGi16]. The common approximation $\frac{1}{2}z(1 + \tanh(\sqrt{2/\pi}(z + 0.044715z^3)))$ is within 0.00047 of the exact GELU on $[-6, 6]$. Both are *non-monotone*: SiLU has a minimum -0.278 at $z = -1.278$, GELU a minimum -0.170 at $z = -0.752$ (found numerically). They behave like ReLU for large $|z|$ but pass small negative signals and have a continuous derivative.

Hidden layers: ReLU for CNNs and MLPs, GELU/SiLU for Transformers, tanh and sigmoid only inside recurrent cells and gates. Output layers: chosen by the task, next section.

§2.2 Output layers: sigmoid and softmax

Prototypical example for this section: Logits $\mathbf{z} = (2, 1, 0.1, -1, 0.5)$ for five classes; the softmax gives class 1 probability 0.5585.

§2.2.i Softmax

For K classes the network outputs **logits** $\mathbf{z} \in \mathbb{R}^K$ and

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}.$$

It is positive and sums to 1, preserves the order of the logits, and is invariant to adding a constant to every logit (multiply top and bottom by e^{-c}), but not to scaling them.

Example 2.2.1 (Softmax confidence from five logits)

For $\mathbf{z} = (2, 1, 0.1, -1, 0.5)$: $e^{\mathbf{z}} = (7.3891, 2.7183, 1.1052, 0.3679, 1.6487)$ with sum 13.2291, so

$$\text{softmax}(\mathbf{z}) = (0.5585, 0.2055, 0.0835, 0.0278, 0.1246).$$

The model predicts class 1 with confidence 55.9%. Note that a logit gap of 1 (2 vs 1) is a probability ratio of $e \approx 2.72$, not a difference of 1 in anything.

§2.2.ii Overflow and the log-sum-exp trick

e^{1000} overflows double precision, so the textbook formula applied to $\mathbf{z} = (1000, 1001, 1002)$ returns *NaN* for every entry. The fix uses shift invariance: subtract $m = \max_k z_k$ first. Then every exponent is ≤ 0 , the largest term is exactly 1, and nothing overflows. The same trick gives a stable **log-sum-exp**

$$\text{LSE}(\mathbf{z}) = \log \sum_k e^{z_k} = m + \log \sum_k e^{z_k - m},$$

which is what a cross-entropy implementation actually needs.

code/dl/c02_activations.py

```

1 def softmax(z):
2     z = z - z.max(axis=-1, keepdims=True)          # shift: same result, no overflow
3     e = np.exp(z)
4     return e / e.sum(axis=-1, keepdims=True)
5
6 def logsumexp(z):
7     m = z.max(axis=-1, keepdims=True)
8     return (m + np.log(np.exp(z - m).sum(axis=-1, keepdims=True))).squeeze(-1)
9
10 def cross_entropy(z, y):
11     # z: (B, K) logits, y: (B,) ints
    return np.mean(logsumexp(z) - z[np.arange(len(y)), y])

```

For (1000, 1001, 1002) the stable version returns (0.090, 0.245, 0.665), identical to $\text{softmax}(0, 1, 2)$, and $\text{LSE} = 1002.4076$; both match PyTorch’s `softmax` and SciPy’s `logsumexp`.

§2.2.iii Temperature

Dividing logits by a **temperature** $T > 0$ before the softmax sharpens ($T < 1$) or flattens ($T > 1$) the distribution without changing the arg-max. For the five logits above (last column: entropy in nats; the uniform distribution has $\ln 5 = 1.609$):

T	p_1	p_2	p_3	p_4	p_5	$H(p)$
0.5	0.826	0.112	0.018	0.002	0.041	0.620
1	0.559	0.205	0.084	0.028	0.125	1.217
2	0.372	0.226	0.144	0.083	0.176	1.495
5	0.264	0.216	0.180	0.145	0.195	1.590

As $T \rightarrow 0$ the softmax becomes a one-hot arg-max, as $T \rightarrow \infty$ it becomes uniform. Temperature reappears in knowledge distillation and LLM sampling ([chapter 9](#)).

§2.2.iv Sigmoid is two-class softmax

$\text{softmax}(a, b)_1 = e^a / (e^a + e^b) = 1 / (1 + e^{-(a-b)}) = \sigma(a - b)$ (checked numerically). So a binary classifier needs one logit and a sigmoid; two logits and a softmax is equivalent but over-parameterised.

§2.2.v The softmax Jacobian

With $\mathbf{p} = \text{softmax}(\mathbf{z})$, $\partial p_i / \partial z_j = p_i(\delta_{ij} - p_j)$, i.e. $J = \text{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^\top$ (verified against `torch.autograd.functional.jacobian`). Rows sum to zero because the outputs always sum to 1. In practice you never form J : combined with cross-entropy it collapses to the gradient $\mathbf{p} - \mathbf{y}$ derived below.

Remark 2.2.2 — Trap. Multi-label is not multi-class. If an image can contain both a cat and a dog, use K independent sigmoids with K binary cross-entropies. A softmax forces the probabilities to compete and sum to 1, so “cat 0.9, dog 0.8” is impossible under it.

§2.3 Loss functions

Prototypical example for this section: True class 1 for the logits above: cross-entropy $-\ln 0.5585 = 0.5824$ nats.

§2.3.i Cross-entropy is maximum likelihood

If the network outputs a distribution $p_\theta(\cdot | \mathbf{x})$ over K classes and the data are independent, the negative log-likelihood of the labels is

$$-\sum_i \log p_\theta(y_i | \mathbf{x}_i) = \sum_i (\text{LSE}(\mathbf{z}_i) - z_{i,y_i}),$$

the **categorical cross-entropy** between the one-hot label and $\mathbf{p}$. Minimising it is minimising the KL divergence from the empirical label distribution to the model (the information-theory section of the companion book *Classical Machine Learning* has the identity $H(q, p) = H(q) + \text{KL}(q||p)$).

Proposition 2.3.1 (Gradient of softmax + cross-entropy)

For one example with logits $\mathbf{z}$, one-hot label $\mathbf{y}$ and $\mathcal{L} = -\sum_k y_k \log \text{softmax}(\mathbf{z})_k$,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}} = \text{softmax}(\mathbf{z}) - \mathbf{y}.$$

Proof. $\mathcal{L} = \text{LSE}(\mathbf{z}) - \mathbf{z}^\top \mathbf{y}$ because $\sum_k y_k = 1$. The gradient of LSE is the softmax, and the gradient of $\mathbf{z}^\top \mathbf{y}$ is $\mathbf{y}$. $\square$

For [example 2.2.1](#) with true class 1 the gradient, identical to autograd's, is

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}} = (-0.4415, 0.2055, 0.0835, 0.0278, 0.1246).$$

The gradient pushes the true logit up by $1 - p_y$ and each wrong logit down by its probability: big mistakes get big corrections, and the gradient never saturates while the prediction is wrong.

Example 2.3.2 (Cross-entropy from logits, many cases)

Each row is checked against `F.cross_entropy`. Classes are numbered from 0 here, as in code.

logits $\mathbf{z}$	y	$\text{softmax}(\mathbf{z})$	pred	p_y	$\text{CE} = -\ln p_y$
(3, 1, 0.2)	0	(0.836, 0.113, 0.051)	0	0.836	0.179
(0, 0, 0, 0)	2	(0.250, 0.250, 0.250, 0.250)	0	0.250	1.386
(2, 2, -1)	1	(0.488, 0.488, 0.024)	0	0.488	0.718
(5, -2, 0, 1)	1	(0.975, 0.001, 0.007, 0.018)	0	0.001	7.026
(1, 2, 3, 4, 5)	0	(0.012, 0.032, 0.086, 0.234, 0.636)	4	0.012	4.452
(-1, 0.5, 0.5, 2.5, 0)	3	(0.022, 0.098, 0.098, 0.723, 0.059)	3	0.723	0.324
(10, 0)	0	(1.000, 0.000)	0	1.000	0.000

Patterns worth memorising: equal logits give $\text{CE} = \ln K$ (row 2); a confident wrong prediction is punished hard (row 4: the true class has probability 0.001); the loss of a correct prediction is small but never exactly 0 (last row); with the same probabilities the loss of predicting the wrong class grows by the logit gap (CE for

class 4 in [example 2.2.1](#) is $3.5824 = 0.5824 + 3$).

Remark 2.3.3 — Trap. Sanity check at initialisation: with small random weights the logits are nearly equal, so the loss of a K -class classifier starts near $\ln K$. For a batch of 64 random 10-class logits of scale 0.01 the script measures 2.3014 against $\ln 10 = 2.3026$. A first loss far from $\ln K$ means a bug (wrong labels, softmax applied twice, a huge initial scale).

§2.3.ii Binary cross-entropy and its stable form

For $y \in \{0, 1\}$ and $p = \sigma(z)$, $\text{BCE} = -[y \log p + (1 - y) \log(1 - p)]$: 0.1054 for $p = 0.9$, $y = 1$; 2.3026 for $p = 0.1$, $y = 1$; $\ln 2 = 0.6931$ at $p = 0.5$; and 4.6052 for a 0.99 prediction of the wrong class. Computing σ first and then log breaks for large $|z|$: at $z = -1000$, $y = 1$ the sigmoid underflows to 0 and the loss becomes ∞ . Working from the logit,

$$\text{BCE}(z, y) = \max(z, 0) - zy + \log(1 + e^{-|z|}),$$

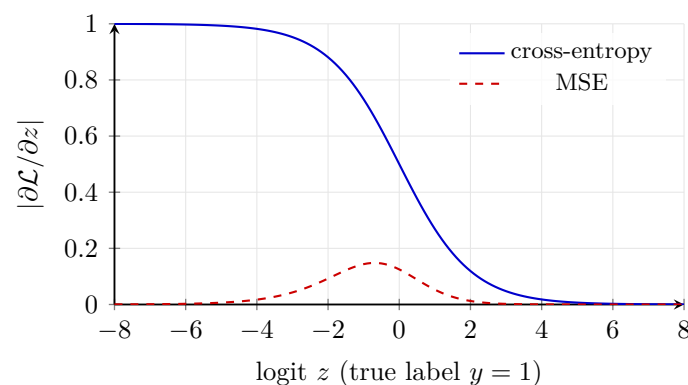
gives the correct 1000.0. This is `BCEWithLogitsLoss`, and the reason you pass logits, not probabilities, to it.

code/dl/c02_activations.py

```
1 def bce_with_logits(z, y):          # = -[y log s(z) + (1-y) log(1-s(z))], stable
2     return np.maximum(z, 0) - z * y + np.log1p(np.exp(-np.abs(z)))
```

§2.3.iii Why not MSE for classification?

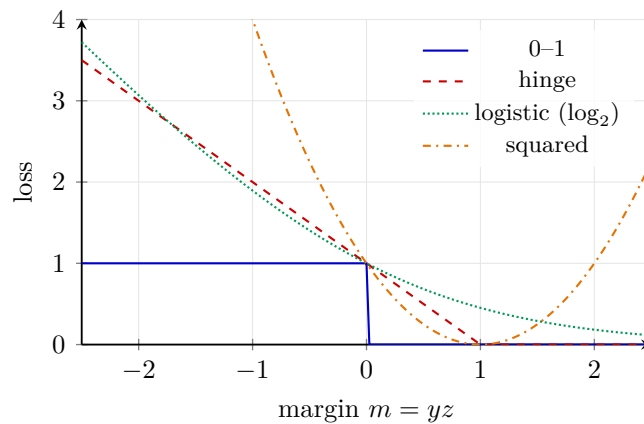
With a sigmoid output and squared error $\frac{1}{2}(\sigma(z) - y)^2$, the gradient is $(\sigma(z) - y) \sigma'(z)$. When the prediction is confidently wrong ($y = 1$, $z = -5$) the factor $\sigma'(z)$ kills it: 0.0066 for MSE against 0.9933 for cross-entropy. The model that is most wrong learns slowest under MSE. Cross-entropy's log exactly cancels the sigmoid's saturation. (MSE on a sigmoid is also non-convex in z ; CE is convex.)



§2.3.iv Margin losses: hinge

With labels $y \in \{-1, +1\}$ and score z , every binary loss is a function of the **margin** $m = yz$. The 0-1 loss $\mathbb{1}[m \leq 0]$ is what we care about but has zero gradient almost everywhere; the **hinge** $\max(0, 1 - m)$ (SVMs) and the logistic loss $\log(1 + e^{-m})$ are convex

upper bounds (the logistic in base 2 so that it passes through $(0, 1)$). The multiclass hinge sums $\max(0, z_j - z_y + 1)$ over wrong classes j ; for [example 2.2.1](#) with true class 2 it is 2.60 (`F.multi_margin_loss` divides by K).



The squared loss punishes *correct* predictions with margin above 1: another reason it is a poor classification loss.

§2.3.v Regression: MSE, MAE, Huber

MSE is the Gaussian negative log-likelihood and is dominated by outliers; MAE (Laplace likelihood) is robust but has a constant-size gradient that does not shrink near the optimum. [Huber](#) interpolates: quadratic within δ , linear outside.

code/dl/c02_activations.py

```
1 def huber(r, delta=1.0):
2     a = np.abs(r)
3     return np.where(a <= delta, 0.5 * r ** 2, delta * (a - 0.5 * delta))
```

With $\delta = 1$, a residual of 10 costs 9.5 under Huber and 50.0 under $\frac{1}{2}r^2$; a residual of 0.2 costs 0.02 under both (matches `F.huber_loss`).

§2.3.vi Focal loss and label smoothing

[Focal loss](#) [Li+17] multiplies cross-entropy by $(1 - p_y)^\gamma$ to down-weight easy, well-classified examples, which dominate the loss when one class (background) vastly outnumbers the rest:

p_y	CE	focal ($\gamma = 2$)	factor $(1 - p_y)^2$
0.9	0.1054	0.0011	0.01
0.5	0.6931	0.1733	0.25
0.1	2.3026	1.8651	0.81

[Label smoothing](#) with parameter ε replaces the one-hot target by $(1 - \varepsilon)\mathbf{y} + \varepsilon/K$; with $\varepsilon = 0.1$, $K = 5$ the target is 0.92 on the true class and 0.02 elsewhere. The model can no longer drive the loss to 0 by pushing one logit to infinity, which curbs over-confidence. For [example 2.2.1](#) the smoothed loss is 0.7304 (`F.cross_entropy(..., label_smoothing=0.1)` agrees).

§2.3.vii Which is a loss, which is an activation?

activations	output maps	losses
ReLU, leaky ReLU, ELU, GELU, SiLU, tanh, sigmoid (hidden)	sigmoid, softmax, identity (linear output)	MSE, MAE, Huber, BCE, CE, hinge, focal, KL divergence, contrastive/triplet

Sigmoid and softmax appear in two columns because they are used both as squashing functions and as output maps; they are never losses.

task	output layer	loss
regression	linear (identity)	MSE / MAE / Huber
binary classification	1 logit, sigmoid	BCE (with logits)
multi-class (one label)	K logits, softmax	categorical CE
multi-label	K logits, K sigmoids	sum of K BCEs
counts	exponential (log-rate)	Poisson NLL

Remark 2.3.4 — Trap. In PyTorch, `nn.CrossEntropyLoss` takes *logits* and applies log-softmax internally. Putting an `nn.Softmax` before it computes softmax twice; training still runs but gradients shrink and the loss floor is wrong. The same holds for `BCEWithLogitsLoss` after a sigmoid (chapter 11 lists this among the silent bugs).

§2.4 A few problems to think about

Problem 2A. A 4-class classifier outputs logits $(1, 1, 1, 1)$. What is the cross-entropy loss? What if the logits are $(101, 101, 101, 101)$?

hint · solution

Problem 2B. Show $\tanh z = 2\sigma(2z) - 1$ and deduce $\tanh'(0) = 4\sigma'(0)$.

hint · solution

Problem 2C. Derive $\partial \text{BCE} / \partial z = \sigma(z) - y$ for $\text{BCE} = -[y \log \sigma(z) + (1 - y) \log(1 - \sigma(z))]$, and verify that the stable form $\max(z, 0) - zy + \log(1 + e^{-|z|})$ equals BCE for both signs of z .

hint · solution

Problem 2D. Which are loss functions? (a) ReLU (b) hinge (c) softmax (d) Huber (e) cross-entropy (f) GELU (g) KL divergence.

hint · solution

Problem 2E. A softmax classifier predicts $p_y = 0.2$ for the true class. By how much must the true logit increase, other logits fixed, to halve the loss? Give a formula, then evaluate for $p_y = 0.2$.

hint · solution

Problem 2F. A team trains an image tagger where each image may have several of 20 tags, with a softmax output and categorical cross-entropy. What goes wrong, and what is the fix?

hint · solution

3 Backpropagation

Backpropagation [RuHiWi86] is the chain rule applied to a computation graph in the order that makes it cheap: from the loss backwards, reusing every intermediate product once. Interviews ask you to derive it for an MLP in matrix form, to run one step by hand on a tiny network, and increasingly to write the autograd engine itself. This chapter does all three, then uses the same algebra to explain why gradients vanish or explode with depth.

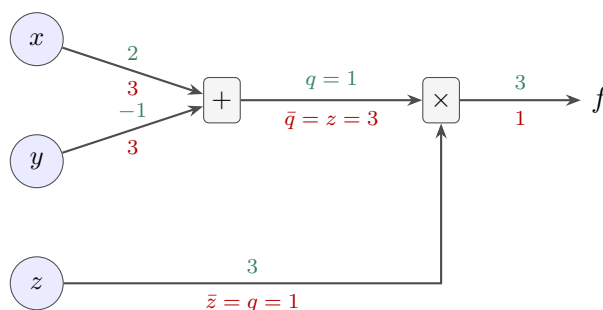
§3.1 Computation graphs and the chain rule

Prototypical example for this section: $f = (x + y)z$ at $x = 2, y = -1, z = 3$: forward gives $q = x + y = 1, f = 3$; backward gives $\partial f / \partial x = \partial f / \partial y = z = 3$ and $\partial f / \partial z = q = 1$.

A **computation graph** is a directed acyclic graph whose nodes are elementary operations and whose edges carry values. To differentiate a scalar output f with respect to every node, define the **adjoint** $\bar{v} = \partial f / \partial v$. For a node v that feeds nodes $u_1, \dots, u_m$,

$$\bar{v} = \sum_{j=1}^m \bar{u}_j \frac{\partial u_j}{\partial v}.$$

Each node needs only its **local gradient** $\partial u / \partial v$; adjoints arrive from above, are multiplied by the local gradient, and are *summed* over all paths (fan-out).



Green numbers are forward values, red are adjoints. The add node *routes* its incoming gradient unchanged to both inputs; the multiply node *swaps*: each input receives the upstream gradient times the other input. A max node routes the whole gradient to the winning input and zero to the rest (that is how max-pooling backpropagates, [chapter 6](#)).

§3.1.i Forward mode versus reverse mode

For $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ the chain rule can be evaluated input-to-output (**forward mode**: one pass per input direction, computing a Jacobian–vector product Jv) or output-to-input (**reverse mode**: one pass per output, computing a vector–Jacobian product $u^\top J$). A loss has $m = 1$ and n in the millions or billions, so reverse mode gets the entire gradient for the cost of about one extra forward pass, while forward mode would need n passes. Backpropagation is reverse-mode automatic differentiation specialised to neural networks.

Remark 3.1.1 — Trap. Backprop is not numerical differentiation and not symbolic differentiation. Numerical differences cost one pass per parameter and are inexact; symbolic expressions blow up in size. Reverse mode is exact up to floating point and costs a small constant times the forward pass, but it must *store* the forward activations, which is why training needs far more memory than inference.

§3.2 Backpropagation for an MLP, in matrix form

Prototypical example for this section: For a ReLU MLP with softmax output, $\delta^{(L)} = \mathbf{p} - \mathbf{y}$ and each layer sends $\delta^{(l)} = (\mathbf{W}^{(l+1)\top} \delta^{(l+1)}) \odot \text{ReLU}'(\mathbf{z}^{(l)})$ back.

Use the notation of chapter 1: $\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}$, $\mathbf{h}^{(l)} = \phi(\mathbf{z}^{(l)})$, $\mathbf{h}^{(0)} = \mathbf{x}$, loss $\mathcal{L}$ on the output logits $\mathbf{z}^{(L)}$. Define the **error signal** $\delta^{(l)} = \partial \mathcal{L} / \partial \mathbf{z}^{(l)}$.

Theorem 3.2.1 (Backpropagation equations)

For one example,

$$\begin{aligned} \delta^{(L)} &= \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(L)}} \quad (= \text{softmax}(\mathbf{z}^{(L)}) - \mathbf{y} \text{ for softmax + CE}), \\ \delta^{(l)} &= (\mathbf{W}^{(l+1)\top} \delta^{(l+1)}) \odot \phi'(\mathbf{z}^{(l)}), \quad l = L-1, \dots, 1, \\ \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} &= \delta^{(l)} \mathbf{h}^{(l-1)\top}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}. \end{aligned}$$

Proof. $z_j^{(l+1)} = \sum_i W_{ji}^{(l+1)} \phi(z_i^{(l)}) + b_j$, so $\partial z_j^{(l+1)} / \partial z_i^{(l)} = W_{ji}^{(l+1)} \phi'(z_i^{(l)})$. The chain rule summed over all j gives $\delta_i^{(l)} = \phi'(z_i^{(l)}) \sum_j W_{ji}^{(l+1)} \delta_j^{(l+1)}$, the second line. Since $z_j^{(l)}$ depends on $W_{ji}^{(l)}$ only through the term $W_{ji}^{(l)} h_i^{(l-1)}$, $\partial \mathcal{L} / \partial W_{ji}^{(l)} = \delta_j^{(l)} h_i^{(l-1)}$, an outer product; the bias has local gradient 1. $\square$

With a batch stored row-wise ($\mathbf{H}^{(l)} \in \mathbb{R}^{B \times n_l}$, $\Delta^{(l)} \in \mathbb{R}^{B \times n_l}$, loss averaged over the batch) the same equations read

$$\Delta^{(l)} = (\Delta^{(l+1)} \mathbf{W}^{(l+1)}) \odot \phi'(\mathbf{Z}^{(l)}), \quad \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \Delta^{(l)\top} \mathbf{H}^{(l-1)}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \Delta^{(l)\top} \mathbf{1},$$

with $\Delta^{(L)} = (\mathbf{P} - \mathbf{Y})/B$. Check the shapes: $\Delta^{(l)\top} \mathbf{H}^{(l-1)}$ is $(n_l, B)(B, n_{l-1}) = (n_l, n_{l-1})$, the shape of $\mathbf{W}^{(l)}$. Shape-matching is the fastest way to rederive these formulas under pressure: there is usually only one way to multiply the available matrices into the required shape.

code/dl/c03_backprop.py

```
1 def mlp_forward_backward(X, y, Ws, bs):
2     """ReLU hidden layers, softmax + cross-entropy output. X: (B, n0), y: (B,) ints
3     ↪ .
4     Ws[l]: (n_{l+1}, n_l) as in nn.Linear. Returns loss and gradients."""
5     B = X.shape[0]
6     hs, zs = [X], []
7     for l, (W, b) in enumerate(zip(Ws, bs)):
8         z = hs[-1] @ W.T + b # (B, n_{l+1})
9         zs.append(z)
10        hs.append(np.maximum(z, 0) if l < len(Ws) - 1 else z)
11    Z = zs[-1] - zs[-1].max(1, keepdims=True)
```

```

11 P = np.exp(Z) / np.exp(Z).sum(1, keepdims=True)
12 loss = -np.mean(np.log(P[np.arange(B), y]))
13 delta = P.copy(); delta[np.arange(B), y] -= 1; delta /= B      # dL/dz_L
14 gWs, gbs = [None] * len(Ws), [None] * len(Ws)
15 for l in reversed(range(len(Ws))):
16     gWs[l] = delta.T @ hs[l]                                # (n_{l+1}, n_l)
17     gbs[l] = delta.sum(0)
18     if l > 0:
19         delta = (delta @ Ws[l]) * (zs[l - 1] > 0) # back through W, then ReLU
20 return loss, gWs, gbs

```

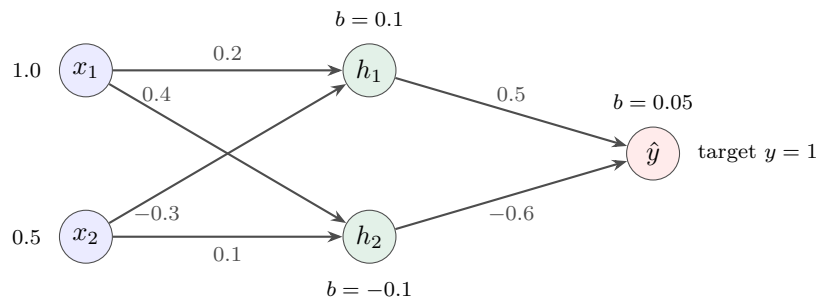
The script runs this on a $5 \rightarrow 8 \rightarrow 6 \rightarrow 3$ network with a batch of 16 and asserts that the loss and all six gradient arrays equal PyTorch's autograd.

layer, forward	local gradient	backward rule
$\mathbf{z} = \mathbf{W}\mathbf{h} + \mathbf{b}$	$\partial \mathbf{z} / \partial \mathbf{h} = \mathbf{W}$	$\bar{\mathbf{h}} = \mathbf{W}^\top \bar{\mathbf{z}}, \bar{\mathbf{W}} = \bar{\mathbf{z}} \mathbf{h}^\top, \bar{\mathbf{b}} = \bar{\mathbf{z}}$
$\mathbf{h} = \phi(\mathbf{z})$ elementwise	$\text{diag } \phi'(\mathbf{z})$	$\bar{\mathbf{z}} = \bar{\mathbf{h}} \odot \phi'(\mathbf{z})$
$\mathbf{h} = \text{ReLU}(\mathbf{z})$	$\mathbb{I}[z > 0]$	$\bar{\mathbf{z}} = \bar{\mathbf{h}} \odot \mathbb{I}[z > 0]$
$s = \sigma(\mathbf{z})$	$s(1 - s)$	$\bar{\mathbf{z}} = \bar{s} s(1 - s)$
$\mathcal{L} = \text{CE}(\text{softmax}(\mathbf{z}), \mathbf{y})$		$\bar{\mathbf{z}} = \mathbf{p} - \mathbf{y}$
$\mathcal{L} = \frac{1}{2} \ \hat{\mathbf{y}} - \mathbf{y}\ ^2$		$\bar{\mathbf{y}} = \hat{\mathbf{y}} - \mathbf{y}$
$\mathbf{C} = \mathbf{A}\mathbf{B}$		$\bar{\mathbf{A}} = \bar{\mathbf{C}}\mathbf{B}^\top, \bar{\mathbf{B}} = \mathbf{A}^\top \bar{\mathbf{C}}$
$u = \max(a, b)$		winner gets $\bar{u}$, loser 0
$u = a + b, u = ab$		route; swap

Backprop is: one forward pass storing $\mathbf{z}^{(l)}$ and $\mathbf{h}^{(l)}$, then for each layer from the top, an outer product for the weight gradient and a transposed matrix product, masked by ϕ' , to pass the error down.

§3.3 A complete forward and backward pass by hand

Prototypical example for this section: A 2-2-1 network with sigmoid hidden units, a sigmoid output and binary cross-entropy, one SGD step with $\eta = 0.5$.



Example 3.3.1 (Forward pass)

$\mathbf{x} = (1, 0.5)$, $\mathbf{W}^{(1)} = \begin{pmatrix} 0.2 & -0.3 \\ 0.4 & 0.1 \end{pmatrix}$, $\mathbf{b}^{(1)} = (0.1, -0.1)$, $\mathbf{w}^{(2)} = (0.5, -0.6)$, $b^{(2)} = 0.05$,

target $y = 1$.

$$\begin{aligned} \mathbf{z}^{(1)} &= \mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)} = (0.1500, 0.3500), & \mathbf{h} &= \sigma(\mathbf{z}^{(1)}) = (0.5374, 0.5866), \\ \mathbf{z}^{(2)} &= \mathbf{w}^{(2)\top} \mathbf{h} + b^{(2)} = -0.0333, & \hat{y} &= \sigma(\mathbf{z}^{(2)}) = 0.4917, \\ \mathcal{L} &= -\ln \hat{y} = 0.7099. \end{aligned}$$

Example 3.3.2 (Backward pass and update)

Output layer: sigmoid + BCE gives $\delta^{(2)} = \hat{y} - y = -0.5083$, so $\partial \mathcal{L} / \partial \mathbf{w}^{(2)} = \delta^{(2)} \mathbf{h} = (-0.2732, -0.2982)$ and $\partial \mathcal{L} / \partial b^{(2)} = -0.5083$.

Hidden layer: $\mathbf{w}^{(2)} \delta^{(2)} = (-0.2542, 0.3050)$, times $\sigma'(\mathbf{z}^{(1)}) = \mathbf{h} \odot (1 - \mathbf{h}) = (0.2486, 0.2425)$ gives $\boldsymbol{\delta}^{(1)} = (-0.0632, 0.0740)$, and

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \boldsymbol{\delta}^{(1)} \mathbf{x}^\top = \begin{pmatrix} -0.0632 & -0.0316 \\ 0.0740 & 0.0370 \end{pmatrix}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(1)}} = (-0.0632, 0.0740).$$

SGD with $\eta = 0.5$: $\mathbf{W}^{(1)} \leftarrow \begin{pmatrix} 0.2316 & -0.2842 \\ 0.3630 & 0.0815 \end{pmatrix}$, $\mathbf{b}^{(1)} \leftarrow (0.1316, -0.1370)$, $\mathbf{w}^{(2)} \leftarrow (0.6366, -0.4509)$, $b^{(2)} \leftarrow 0.3042$. The loss on the same example drops from 0.7099 to 0.5122. Every gradient matches `loss.backward()` in PyTorch.

code/dl/c03_backprop.py

```
1 # forward
2 z1 = W1 @ x0 + b1          # (2,)
3 h1 = sig(z1)               # hidden activations
4 z2 = W2 @ h1 + b2          # scalar logit
5 yhat = sig(z2)
6 L = -(y0 * np.log(yhat) + (1 - y0) * np.log(1 - yhat)) # BCE
7 # backward
8 d2 = yhat - y0             # dL/dz2 (sigmoid + BCE)
9 gW2 = d2 * h1; gb2 = d2
10 d1 = (W2 * d2) * h1 * (1 - h1) # dL/dz1
11 gW1 = np.outer(d1, x0); gb1 = d1
12 # one SGD step
13 W1n, b1n, W2n, b2n = W1 - eta * gW1, b1 - eta * gb1, W2 - eta * gW2, b2 - eta * gb2
```

Two things to notice. The hidden unit with the *negative* outgoing weight gets a positive δ : increasing its activation would lower $\hat{y}$, so its weights are pushed down. And the update to $\mathbf{W}^{(1)}$ is an outer product, rank one: one example moves the first layer only along the direction of its own input $\mathbf{x}$.

Remark 3.3.3 — Trap. With MSE $\frac{1}{2}(\hat{y} - y)^2$ instead of BCE the output error picks up an extra factor: $\delta^{(2)} = (\hat{y} - y) \hat{y}(1 - \hat{y})$. Forgetting which loss the question uses is the most common way to get this OA wrong; the clean $\hat{y} - y$ is specific to sigmoid + BCE (and softmax + CE).

§3.4 Gradient checking

Prototypical example for this section: Compare the analytic $\partial \mathcal{L} / \partial \mathbf{W}^{(1)}$ with $(\mathcal{L}(\theta + h\mathbf{e}_i) - \mathcal{L}(\theta - h\mathbf{e}_i)) / 2h$; a relative error near 10^{-9} means the backward pass is right.

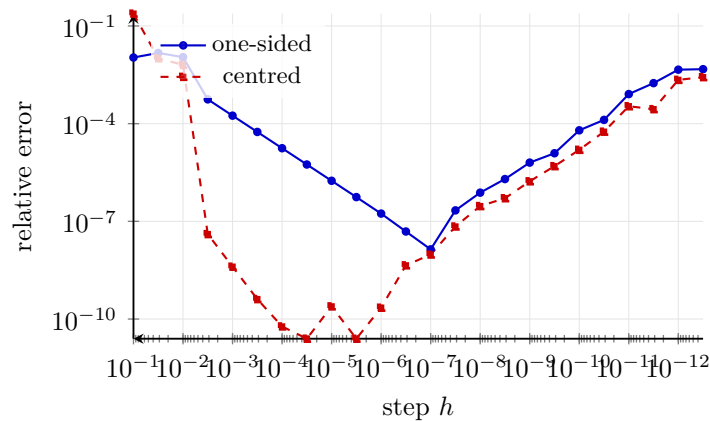
code/dl/c03_backprop.py

```

1 def numerical_grad(f, theta, h=1e-5):
2     g = np.zeros_like(theta)
3     for i in range(theta.size):
4         e = np.zeros_like(theta); e.flat[i] = h
5         g.flat[i] = (f(theta + e) - f(theta - e)) / (2 * h)      # centred
6         ↪ difference
7     return g
8
9 def rel_error(a, b):
10    return np.max(np.abs(a - b) / np.maximum(1e-12, np.abs(a) + np.abs(b)))

```

The centred difference has truncation error $O(h^2)$, the one-sided difference $O(h)$; both suffer round-off error $O(\epsilon_{\text{mach}}/h)$. So the error is U-shaped in h :



In double precision the centred difference is best near $h \approx 3 \times 10^{-6}$ (error 2×10^{-11}), the one-sided near $h \approx 1 \times 10^{-7}$ (error 1×10^{-8}). With $h = 10^{-5}$ the full check on $\mathbf{W}^{(1)}$ gives relative error 4.4×10^{-9} . Plant a bug, dropping the ReLU mask in the backward pass, and the relative error jumps to 1.00.

Remark 3.4.1 — Trap. Gradient checks in `float32` fail spuriously: round-off is then about $10^{-7}/h$, so use `float64` for the check. Also turn off dropout and anything random (the two loss evaluations must see the same mask), and avoid points where ReLUs sit exactly at their kink.

§3.5 Reverse-mode autograd from scratch

Prototypical example for this section: A `Tensor` class that records parents and a backward closure; calling `loss.backward()` visits the graph in reverse topological order.

The whole of an autograd engine is three ideas: each operation returns a new node that remembers its inputs and a function mapping the output's gradient to the inputs' gradients; a topological sort; and accumulation (`+=`) for nodes used more than once. Broadcasting needs one more line: the gradient of a broadcast input is summed over the broadcast dimensions.

code/dl/c03_backprop.py

```

1 class Tensor:
2     def __init__(self, data, parents=(), backward=lambda g: ()):
3         self.data = np.asarray(data, dtype=float)

```

```

4     self.grad = np.zeros_like(self.data)
5     self.parents, self._backward = parents, backward
6
7     @staticmethod
8     def _unbroadcast(g, shape):          # sum gradient over broadcast dims
9         while g.ndim > len(shape):
10             g = g.sum(0)
11         for i, s in enumerate(shape):
12             if s == 1:
13                 g = g.sum(i, keepdims=True)
14         return g
15
16     def __add__(self, o):
17         o = o if isinstance(o, Tensor) else Tensor(o)
18         return Tensor(self.data + o.data, (self, o),
19                       lambda g: (self._unbroadcast(g, self.data.shape),
20                                 self._unbroadcast(g, o.data.shape)))
21
22     def __mul__(self, o):
23         o = o if isinstance(o, Tensor) else Tensor(o)
24         return Tensor(self.data * o.data, (self, o),
25                       lambda g: (self._unbroadcast(g * o.data, self.data.shape),
26                                 self._unbroadcast(g * self.data, o.data.shape)))
27
28     def __matmul__(self, o):
29         return Tensor(self.data @ o.data, (self, o),
30                       lambda g: (g @ o.data.T, self.data.T @ g))
31
32     def relu(self):
33         return Tensor(np.maximum(self.data, 0), (self,), lambda g: (g * (self.data
34 ↪ > 0),))
35
36     def tanh(self):
37         t = np.tanh(self.data)
38         return Tensor(t, (self,), lambda g: (g * (1 - t ** 2),))
39
40     def sum(self):
41         return Tensor(self.data.sum(), (self,), lambda g: (g * np.ones_like(self.
42 ↪ data),))
43
44     def cross_entropy(self, y):          # logits (B, K), integer labels (B,)
45         Z = self.data - self.data.max(1, keepdims=True)
46         P = np.exp(Z) / np.exp(Z).sum(1, keepdims=True)
47         B = len(y)
48         loss = -np.log(P[np.arange(B), y]).mean()
49         def back(g):
50             d = P.copy(); d[np.arange(B), y] -= 1
51             return (g * d / B,)
52         return Tensor(loss, (self,), back)
53
54     def backward(self):
55         order, seen = [], set()
56         def visit(t):                    # topological order by DFS
57             if id(t) not in seen:
58                 seen.add(id(t))
59                 for p in t.parents:
60                     visit(p)
61                 order.append(t)
62         visit(self)
63         self.grad = np.ones_like(self.data)
64         for t in reversed(order):        # outputs before inputs

```

```

63         for p, g in zip(t.parents, t._backward(t.grad)):
64             p.grad = p.grad + g

```

That is 55 non-blank lines. The script builds a ReLU–tanh network with a broadcast bias and a term that uses $\mathbf{h}$ twice ($\mathbf{h} \odot \mathbf{h}$) plus a broadcast scalar, and asserts that the loss and the gradients of all six parameter arrays equal PyTorch’s.

Question 3.5.1. Why does `backward` initialise the output’s gradient to one, and what would `loss.backward()` mean in PyTorch if `loss` were a vector?

§3.6 Vanishing and exploding gradients

Prototypical example for this section: In a 30-layer sigmoid MLP, the gradient reaching the first layer is 5.4×10^{-19} times the one at the last layer.

§3.6.i The product of Jacobians

Unrolling the recursion,

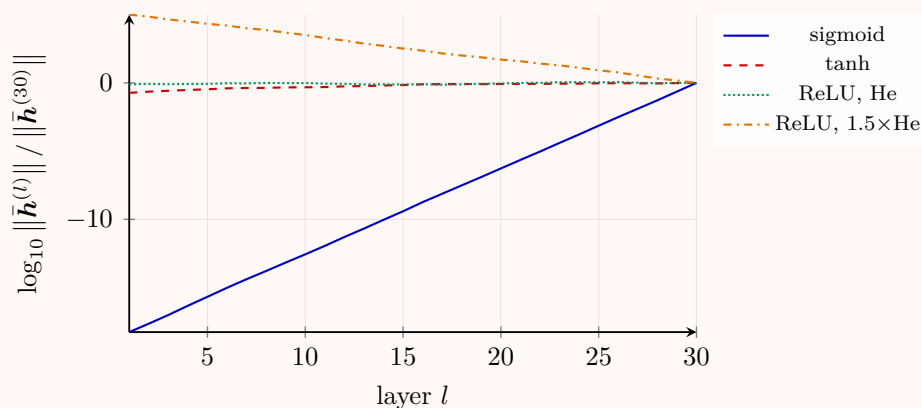
$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(l)}} = \left(\prod_{k=l+1}^L \mathbf{W}^{(k)\top} \text{diag} \phi'(z^{(k)}) \right) \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(L)}},$$

with the factor for $k = l + 1$ on the left. The norm of a product of $L - l$ random matrices behaves roughly like ρ^{L-l} for some per-layer gain ρ . If $\rho < 1$ the gradient **vanishes** exponentially with depth; if $\rho > 1$ it **explodes**. With sigmoids, $\phi' \leq \frac{1}{4}$ forces ρ well below one unless the weights are large, and large weights saturate the sigmoids, making ϕ' even smaller.

Example 3.6.1 (Gradient norm through depth)

A 30-layer, width-64 network on Gaussian inputs, no biases, gradient of a random linear readout. Ratio of the gradient norm at layer 1 to that at layer 30:

activation	weight init (std)	$\ \bar{\mathbf{h}}^{(1)}\ / \ \bar{\mathbf{h}}^{(30)}\ $
sigmoid	$\sqrt{1/n}$	5.4×10^{-19}
tanh	$\sqrt{1/n}$	1.9×10^{-1}
ReLU	$\sqrt{2/n}$ (He)	7.9×10^{-1}
ReLU	$1.5\sqrt{2/n}$	1.0×10^5



A $1.5\times$ too large initial scale per layer compounds to five orders of magnitude over 30 layers: initialisation is not a detail (chapter 5).

The fixes, each covered later: activations with $\phi' = 1$ on the active side (ReLU family); variance-preserving initialisation (Xavier, He); normalisation layers (BatchNorm, LayerNorm); residual connections, which add an identity term to every Jacobian so the product cannot collapse (ResNet, chapter 6); gating in LSTMs (chapter 7); and gradient clipping for the exploding case (chapter 4).

Remark 3.6.2 — Trap. Vanishing gradients slow down learning in the *early* layers (those closest to the input), not the late ones. The last layers train fine; the network then behaves like a shallow model on top of nearly random features.

§3.6.ii Dying ReLUs

A ReLU unit with $z < 0$ for every input outputs 0 and passes back 0, so its incoming weights never change again: it is **dead**. Two ways to kill units, both simulated in `code/dl/c03_backprop.py`: a large learning-rate step that throws biases far negative, and a bad initialisation. Training a width-200 ReLU layer on a regression task for 300 SGD steps:

learning rate	dead units	final training MSE
0.01	0.0%	0.71
0.1	0.0%	0.02
0.2	68.0%	15.00

Initialising all biases at -3 leaves 99.0% of units dead before training starts. Remedies: a smaller learning rate or warm-up, leaky ReLU/ELU/GELU, He initialisation with zero biases, normalisation before the ReLU.

§3.7 What backprop costs

Prototypical example for this section: A $784 \rightarrow 256 \rightarrow 128 \rightarrow 10$ MLP: 234,752 multiply-adds per example forward, about three times that per training step.

A dense layer with n_{in} inputs and n_{out} outputs costs $n_{\text{in}}n_{\text{out}}$ multiply-accumulates (MACs), i.e. $2n_{\text{in}}n_{\text{out}}$ FLOPs, per example. The backward pass needs two products of the same size, $\bar{\mathbf{h}} = \mathbf{W}^\top \bar{\mathbf{z}}$ and $\bar{\mathbf{W}} = \bar{\mathbf{z}}\mathbf{h}^\top$, so a training step costs about $3\times$ the forward pass: for the MNIST MLP, 469,504 FLOPs forward and about 1,408,512 per example per step. (The first layer's $\bar{\mathbf{h}} = \mathbf{W}^\top \bar{\mathbf{z}}$ can be skipped since nobody needs $\partial\mathcal{L}/\partial\mathbf{x}$.)

Memory is the other bill: every $\mathbf{z}^{(l)}$ or $\mathbf{h}^{(l)}$ needed by a backward rule is kept from the forward pass, so activation memory grows with $\text{depth} \times \text{batch size} \times \text{width}$. **Gradient checkpointing** stores only some layers' activations and recomputes the rest during the backward pass, trading about one extra forward pass for memory that grows like $\sqrt{L}$ instead of L .

§3.8 A few problems to think about

Problem 3A. For $f = \max(x, y) \cdot z$ at $x = 1$, $y = 4$, $z = -2$, give $\partial f / \partial x$, $\partial f / \partial y$, $\partial f / \partial z$.

hint · solution



Problem 3B. A linear layer maps a batch $\mathbf{X} \in \mathbb{R}^{32 \times 100}$ to $\mathbf{X}\mathbf{W}^\top + \mathbf{b}$ with $\mathbf{W} \in \mathbb{R}^{50 \times 100}$. What are the shapes of $\partial\mathcal{L}/\partial\mathbf{W}$, $\partial\mathcal{L}/\partial\mathbf{b}$ and $\partial\mathcal{L}/\partial\mathbf{X}$, and which products give them?

[hint](#) · [solution](#)

Problem 3C. Redo [example 3.3.2](#) with the loss $\frac{1}{2}(\hat{y} - y)^2$: give $\delta^{(2)}$ symbolically and say whether the hidden-layer updates get larger or smaller than with BCE.

[hint](#) · [solution](#)

Problem 3D. Show that if all weights of an MLP are initialised to the same constant (and all biases equal), every hidden unit in a layer stays identical throughout training. Do the output units stay identical too?

[hint](#) · [solution](#)

Problem 3E. Explain, with the product-of-Jacobians formula, why a residual block $\mathbf{h}^{(l+1)} = \mathbf{h}^{(l)} + F(\mathbf{h}^{(l)})$ helps gradients reach early layers.

[hint](#) · [solution](#)

Problem 3F. Training loss stays exactly constant from the first step in a ReLU network. Give two causes related to this chapter and a check for each.

[hint](#) · [solution](#)

4 Optimisers and learning-rate schedules

Backpropagation gives the gradient; the optimiser decides what to do with it. The OA version of this chapter is “what is the purpose of Adam?” and one hand-computed update. The interview version is why plain gradient descent zig-zags, what momentum and Adam’s second moment fix, why Adam’s bias correction exists, and why the learning-rate schedule matters as much as the optimiser.

§4.1 Gradient descent and its one big problem

Prototypical example for this section: On $f(x, y) = \frac{1}{2}(x^2 + 10y^2)$ gradient descent must use a step below $2/10$, so progress along x is ten times slower than it could be.

§4.1.i Curvature limits the step

Gradient descent updates $\theta \leftarrow \theta - \eta \nabla \mathcal{L}(\theta)$. Near a minimum the loss is approximately quadratic, $\mathcal{L}(\theta) \approx \mathcal{L}^* + \frac{1}{2}(\theta - \theta^*)^\top \mathbf{H}(\theta - \theta^*)$, and in the eigenbasis of the Hessian $\mathbf{H}$ each coordinate evolves independently: $e_i \leftarrow (1 - \eta \lambda_i) e_i$. Convergence needs $|1 - \eta \lambda_i| < 1$ for all i , i.e. $\eta < 2/\lambda_{\max}$; the slowest direction then shrinks by $1 - \eta \lambda_{\min}$ per step. The best fixed step, $\eta = 2/(\lambda_{\min} + \lambda_{\max})$, gives the contraction factor

$$\frac{\kappa - 1}{\kappa + 1}, \quad \kappa = \frac{\lambda_{\max}}{\lambda_{\min}} \quad (\text{the condition number}).$$

Example 4.1.1 (An ill-conditioned bowl)

For $\mathbf{H} = \text{diag}(1, 10)$: steps above 0.20 diverge (with $\eta = 0.21$, $|y|$ grows to 6.7 in 20 steps from 1). The optimal step $\eta = 0.1818$ contracts the error by 0.8182 per step (the script measures exactly this), so reducing it by 10^6 takes 69 steps. Heavy-ball momentum with its optimal parameters contracts by $(\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1) = 0.5195$ and needs 22 steps.

The geometric picture: the gradient points across a narrow valley rather than along it, so a step large enough to make progress along the valley overshoots across it. Deep-network losses have Hessians with condition numbers in the thousands or worse, which is why every optimiser in this chapter is, one way or another, a fix for ill-conditioning.

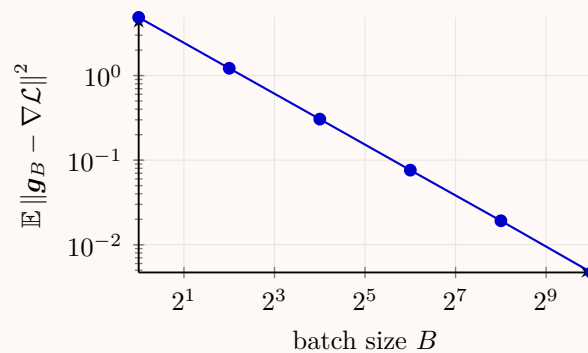
§4.1.ii Stochastic and minibatch gradients

The training loss is an average over N examples. A minibatch gradient over B random examples is an unbiased estimate of the full gradient whose variance falls like $1/B$: averaging B independent terms divides the variance by B .

Example 4.1.2 (Gradient noise against batch size)

For logistic regression on 20,000 synthetic points, the mean squared deviation of a minibatch gradient from the full gradient, as a function of B , has log-log slope

−1.001; going from $B = 1$ to $B = 64$ cuts it by a factor of 63.9.



Vocabulary that OAs test: an **iteration** (step) processes one minibatch; an **epoch** is one pass over the training set. With $N = 50000$ and $B = 128$ an epoch is $\lceil N/B \rceil = 391$ iterations (the last batch is partial unless dropped), so 30 epochs are 11730 steps. “SGD” in deep learning almost always means minibatch SGD.

§4.2 Momentum and Nesterov

Prototypical example for this section: A ball rolling down the valley accumulates speed along the floor while the side-to-side bounces cancel.

Momentum keeps a running sum of past gradients:

$$\mathbf{v} \leftarrow \beta \mathbf{v} + \mathbf{g}, \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \mathbf{v} \quad (\text{PyTorch convention}).$$

Components of the gradient that keep their sign add up (on a constant slope $\mathbf{v} \rightarrow \mathbf{g}/(1-\beta)$, so $\beta = 0.9$ is a $10\times$ larger effective step), while components that flip sign every step cancel. That is exactly the valley geometry above. **Nesterov momentum** evaluates the gradient at the look-ahead point $\boldsymbol{\theta} - \eta \beta \mathbf{v}$; in PyTorch’s equivalent form the update uses $\mathbf{g} + \beta \mathbf{v}$ instead of $\mathbf{v}$. It reacts one step earlier when the velocity is about to overshoot.

Example 4.2.1 (Two momentum steps)

$\eta = 0.1$, $\beta = 0.9$, $\theta_0 = 1$, gradients 2 then 1: $v_1 = 2.00$, $\theta_1 = 0.80$; $v_2 = 0.9 \cdot 2 + 1 = 2.80$, $\theta_2 = 0.52$. The second gradient is half the first, yet the second step is larger.

Remark 4.2.2 — Trap. Two conventions exist. Many texts write $\mathbf{v} \leftarrow \beta \mathbf{v} + (1-\beta)\mathbf{g}$ (an exponential moving average); PyTorch’s SGD uses $\mathbf{v} \leftarrow \beta \mathbf{v} + \mathbf{g}$ (its dampening defaults to 0). For the same η the PyTorch step is $1/(1-\beta)$ times larger, so learning rates do not transfer between the two without rescaling.

§4.3 Adaptive methods: AdaGrad, RMSProp, Adam

Prototypical example for this section: Adam’s first step moves every parameter by almost exactly η , whether its gradient is 0.5 or -0.02 .

§4.3.i Per-parameter step sizes

Momentum still uses one step size for all parameters. Adaptive methods divide each coordinate by a running estimate of its gradient scale, which approximately rescales the valley into a round bowl when the Hessian is close to diagonal:

- **AdaGrad**: $G \leftarrow G + \mathbf{g}^2$, $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \mathbf{g} / (\sqrt{G} + \varepsilon)$. Rare features (sparse gradients) get large steps; but G only grows, so the step size decays to zero, too early for deep networks.
- **RMSProp**: replace the sum by an exponential moving average, $\mathbf{v} \leftarrow \alpha \mathbf{v} + (1 - \alpha) \mathbf{g}^2$, so the scale tracks recent gradients and the step does not die.
- **Adam** [KiBa15] = momentum on $\mathbf{g}$ plus RMSProp on $\mathbf{g}^2$, with bias correction:

Algorithm: Adam, step t (defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$)

$$\begin{aligned} \mathbf{m}_t &= \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, & \mathbf{v}_t &= \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2, \\ \hat{\mathbf{m}}_t &= \mathbf{m}_t / (1 - \beta_1^t), & \hat{\mathbf{v}}_t &= \mathbf{v}_t / (1 - \beta_2^t), \\ \boldsymbol{\theta}_t &= \boldsymbol{\theta}_{t-1} - \eta \hat{\mathbf{m}}_t / (\sqrt{\hat{\mathbf{v}}_t} + \varepsilon). \end{aligned}$$

§4.3.ii Why bias correction

With $\mathbf{m}_0 = \mathbf{0}$, $\mathbf{m}_t = (1 - \beta_1) \sum_{s=1}^t \beta_1^{t-s} \mathbf{g}_s$. If the gradients have constant mean $\mathbf{g}$, then $\mathbb{E} \mathbf{m}_t = (1 - \beta_1^t) \mathbf{g}$: the average is biased toward the zero initialisation by exactly the factor $1 - \beta_1^t$, which dividing removes. The bias matters most for $\mathbf{v}$, because $1 - \beta_2^t$ stays small for hundreds of steps ($1 - 0.999^t \approx t/1000$). Without correction the first step would be $\eta \cdot 0.1 \mathbf{g} / \sqrt{0.001 \mathbf{g}^2} = \eta \cdot 0.1 / \sqrt{0.001} \approx 3.16 \eta$; for $\eta = 10^{-3}$ that is 3.16×10^{-3} , a too-large first step taken at the moment the estimates are least reliable.

Example 4.3.1 (Two Adam steps by hand)

Two parameters at $\boldsymbol{\theta}_0 = (1, 1)$, $\eta = 10^{-3}$, default betas, gradients $\mathbf{g}_1 = (0.5, -0.02)$ then $\mathbf{g}_2 = (0.3, 0.04)$.

Step 1. $\mathbf{m}_1 = 0.1 \mathbf{g}_1 = (0.0500, -0.0020)$, $\mathbf{v}_1 = 0.001 \mathbf{g}_1^2 = (2.50 \times 10^{-4}, 4.00 \times 10^{-7})$. Bias correction divides by 0.1 and 0.001, giving $\hat{\mathbf{m}}_1 = \mathbf{g}_1$, $\hat{\mathbf{v}}_1 = \mathbf{g}_1^2$, so the step is $\eta \mathbf{g}_1 / |\mathbf{g}_1|$: both parameters move by $(1.000 \times 10^{-3}, -1.000 \times 10^{-3})$, one down and one up. The 25-fold difference in gradient size has disappeared.

Step 2. $\mathbf{m}_2 = 0.9 \mathbf{m}_1 + 0.1 \mathbf{g}_2 = (0.0750, 0.0022)$, $\mathbf{v}_2 = 0.999 \mathbf{v}_1 + 0.001 \mathbf{g}_2^2 = (3.398 \times 10^{-4}, 2.000 \times 10^{-6})$. Corrections $1 - 0.9^2 = 0.19$ and $1 - 0.999^2 = 0.001999$ give $\hat{\mathbf{m}}_2 = (0.3947, 0.0116)$ and $\hat{\mathbf{v}}_2 = (1.700 \times 10^{-1}, 1.000 \times 10^{-3})$; the parameters decrease by $(9.575 \times 10^{-4}, 3.661 \times 10^{-4})$, ending at $\boldsymbol{\theta}_2 = (0.998043, 1.000634)$, exactly as `torch.optim.Adam`. The second parameter's gradient changed sign, so its averaged momentum is small relative to its RMS and it moves only about a third of η .

code/dl/c04_optim.py

```
1 class SGD:
2     def __init__(self, lr, momentum=0.0, nesterov=False, weight_decay=0.0):
3         self.lr, self.mu, self.nesterov, self.wd, self.buf = lr, momentum, nesterov
```

```

    ↪ , weight_decay, None
4  def step(self, p, g):
5      g = g + self.wd * p                                # L2 penalty folded into g
6      if self.mu:
7          self.buf = g if self.buf is None else self.mu * self.buf + g
8          g = g + self.mu * self.buf if self.nesterov else self.buf
9      return p - self.lr * g
10
11 class AdaGrad:
12     def __init__(self, lr, eps=1e-10):
13         self.lr, self.eps, self.G = lr, eps, 0.0
14     def step(self, p, g):
15         self.G = self.G + g * g                        # sum of all squared gradients
16         return p - self.lr * g / (np.sqrt(self.G) + self.eps)
17
18 class RMSProp:
19     def __init__(self, lr, alpha=0.99, eps=1e-8):
20         self.lr, self.a, self.eps, self.v = lr, alpha, eps, 0.0
21     def step(self, p, g):
22         self.v = self.a * self.v + (1 - self.a) * g * g # EMA of squared
    ↪ gradients
23         return p - self.lr * g / (np.sqrt(self.v) + self.eps)
24
25 class Adam:
26     def __init__(self, lr, betas=(0.9, 0.999), eps=1e-8, weight_decay=0.0,
    ↪ decoupled=False):
27         self.lr, (self.b1, self.b2), self.eps = lr, betas, eps
28         self.wd, self.decoupled, self.m, self.v, self.t = weight_decay, decoupled,
    ↪ 0.0, 0.0, 0
29     def step(self, p, g):
30         self.t += 1
31         if self.decoupled:                                # AdamW: decay the weights
    ↪ directly
32             p = p * (1 - self.lr * self.wd)
33         else:                                              # Adam + L2: decay enters the
    ↪ gradient
34             g = g + self.wd * p
35             self.m = self.b1 * self.m + (1 - self.b1) * g
36             self.v = self.b2 * self.v + (1 - self.b2) * g * g
37             m_hat = self.m / (1 - self.b1 ** self.t)      # bias correction
38             v_hat = self.v / (1 - self.b2 ** self.t)
39             return p - self.lr * m_hat / (np.sqrt(v_hat) + self.eps)

```

All nine variants above (SGD, momentum, Nesterov, SGD with weight decay, AdaGrad, RMSProp, Adam, Adam with L2, AdamW) are run for 25 steps on a non-quadratic test function and their trajectories asserted equal to `torch.optim`'s.

Remark 4.3.2 — Trap. “What is the purpose of Adam?” is not “to add momentum” and not “to reduce overfitting”. Adam gives each parameter its own effective learning rate, scaled by the inverse RMS of its recent gradients, and combines that with momentum; the practical effect is fast, robust progress with little learning-rate tuning. It does not regularise.

Remark 4.3.3 — Trap. “Adam always beats SGD.” On many image-classification benchmarks well-tuned SGD with momentum reaches equal or better test accuracy; Adam’s coordinate-wise rescaling can settle in different (often sharper) solutions,

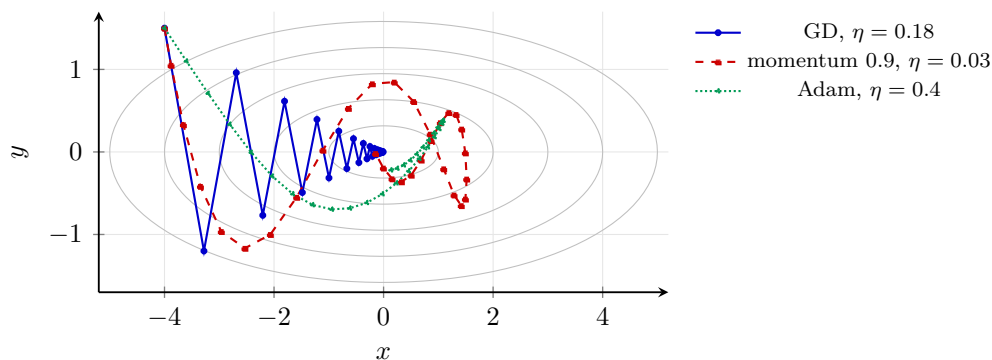
and with L2 regularisation it under-decays parameters with large gradient history. Adam and especially AdamW dominate for Transformers. The honest interview answer is “it depends on the architecture, and the gap is usually closed by tuning the schedule and weight decay”.

§4.3.iii AdamW: weight decay done properly

With plain SGD, adding $\frac{\lambda}{2} \|\theta\|^2$ to the loss and shrinking the weights by $(1 - \eta\lambda)$ each step are the same thing. With Adam they are not: the L2 gradient $\lambda\theta$ is divided by $\sqrt{\hat{v}}$ like everything else, so parameters with large gradients are barely decayed. **AdamW** [LoHu19] decouples the decay, $\theta \leftarrow \theta(1 - \eta\lambda)$, outside the adaptive step (the **decoupled** branch in the code). Chapter 5 measures the difference.

§4.4 Paths and curves

Prototypical example for this section: Thirty steps of GD, momentum and Adam from $(-4, 1.5)$ on the bowl of the first section.

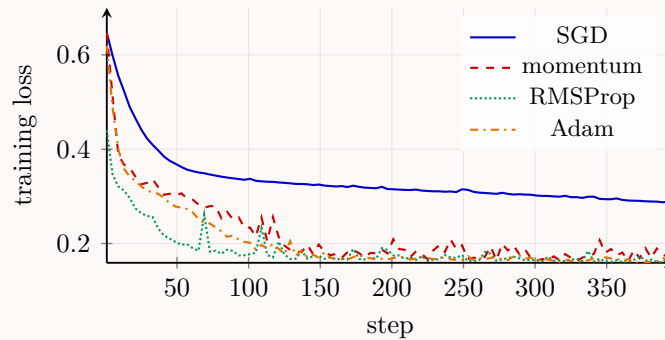


GD with a step close to the stability limit zig-zags across the narrow direction; momentum overshoots along the valley and spirals in; Adam moves at roughly constant speed η per coordinate and, with a fixed learning rate, keeps hovering around the minimum (final loss 0.2504 against 0.0001 for GD). That last behaviour is the argument for decaying the learning rate.

Example 4.4.1 (Training curves on a real network)

A 2-64-64-2 ReLU network on a noisy two-moons dataset, minibatches of 64, the same initialisation for every optimiser; full-data training loss after 100 and 400 steps:

optimiser	step 100	step 400
SGD, $\eta = 0.05$	0.337	0.286
SGD + momentum 0.9	0.223	0.180
RMSPProp, $\eta = 0.003$	0.178	0.161
Adam, $\eta = 0.003$	0.201	0.159



One run on one problem proves nothing general; the point is the shape (momentum and the adaptive methods leave plain SGD behind early) and how to set up a fair comparison: same initialisation, same batches, each optimiser at a sensible learning rate.

§4.5 Learning-rate schedules

Prototypical example for this section: Cosine decay from 0.1 over 100 epochs: 0.0854 at epoch 25, 0.0500 at epoch 50, 0.0146 at epoch 75.

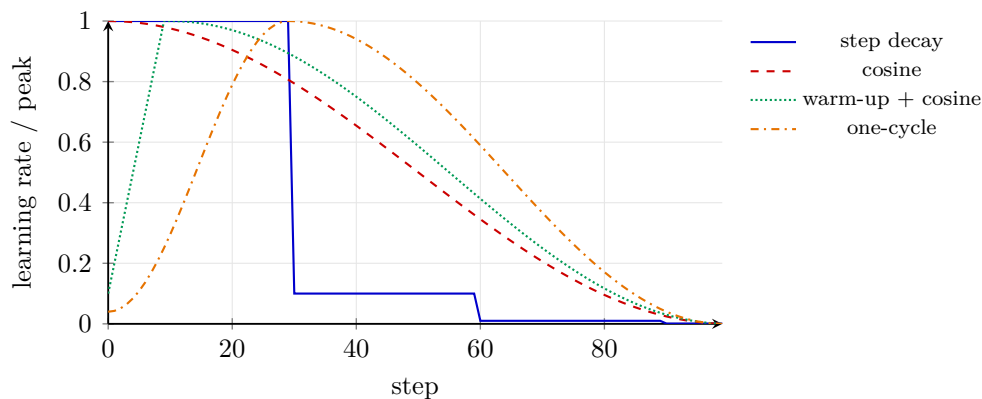
A large learning rate early makes fast progress and adds useful noise; a small one late lets the iterate settle into a minimum (recall Adam hovering above). Schedules interpolate:

code/dl/c04_optim.py

```

1 def step_decay(t, lr0, step=30, gamma=0.1):
2     return lr0 * gamma ** (t // step)
3
4 def cosine(t, lr0, T, lr_min=0.0):
5     return lr_min + 0.5 * (lr0 - lr_min) * (1 + math.cos(math.pi * t / T))
6
7 def warmup_cosine(t, lr0, W, T):
8     if t < W:
9         return lr0 * (t + 1) / W # linear warm-up
10    return cosine(t - W, lr0, T - W)
11
12 def one_cycle(t, lr_max, T, pct=0.3, div=25.0, final_div=1e4):
13     up = pct * T - 1
14     lo, end = lr_max / div, lr_max / div / final_div
15     cos_anneal = lambda a, b, frac: b + (a - b) / 2 * (1 + math.cos(math.pi * frac)
16     ⇐⇒ )
17     if t <= up:
18         return cos_anneal(lo, lr_max, t / up)
19     return cos_anneal(lr_max, end, (t - up) / (T - 1 - up))

```



The script asserts that step decay, cosine and one-cycle reproduce the schedulers `StepLR`, `CosineAnnealingLR` and `OneCycleLR` of PyTorch exactly. Step decay with $\eta_0 = 0.1$, $\gamma = 0.1$ every 30 epochs gives 0.0010 at epoch 65.

Warm-up ramps the learning rate up linearly over the first few hundred or thousand steps. Early on, the weights are random, activations and gradients can be large and Adam’s second-moment estimate rests on a handful of samples, so a full-size step is risky; Transformers in particular often diverge without warm-up. **One-cycle** goes up then down in a single cycle (with momentum moving the opposite way) and is popular for short training budgets.

Remark 4.5.1 — Trap. Schedulers in PyTorch are stepped explicitly. Calling `scheduler.step()` once per batch for a schedule defined in epochs (or the reverse) silently compresses or stretches it by a factor of the number of batches per epoch.

§4.6 Batch size, clipping and the loss landscape

Prototypical example for this section: Doubling the batch halves the gradient variance, so the usual heuristic doubles the learning rate with it.

§4.6.i Batch size

Because the gradient noise variance scales like $1/B$, a larger batch gives a cleaner gradient and allows a proportionally larger step: the **linear scaling heuristic** ($\eta \propto B$, used with warm-up) works up to a problem-dependent critical batch size, beyond which larger batches stop reducing the number of steps needed. Small batches inject more noise, which is widely reported to help generalisation; the explanations (noise steering toward flatter minima) are plausible but not settled, so state them as such. Practical constraints usually decide B : memory, and hardware throughput.

§4.6.ii Gradient clipping

Clipping by norm rescales the whole gradient when its norm exceeds a threshold c : $\mathbf{g} \leftarrow \mathbf{g} \cdot \min(1, c/\|\mathbf{g}\|)$. For $\mathbf{g} = (3, 4)$ and $c = 1$ it gives $(0.6, 0.8)$: same direction, length 1. **Clipping by value** clips each coordinate to $[-c, c]$, giving $(1, 1)$, which changes the direction. Both match PyTorch’s `clip_grad_norm_` and `clip_grad_value_`. Clipping by norm is the standard fix for exploding gradients in RNNs and is routine in Transformer training; it does nothing for vanishing gradients.

§4.6.iii Loss landscapes

In high dimension most critical points of a neural-network loss are **saddle points**: the Hessian has both positive and negative eigenvalues, and for a random critical point it is unlikely that all of thousands of eigenvalues are positive. Gradient descent slows down near saddles (the gradient is small) but stochastic noise and momentum push it off them. Poor local minima are less of a practical problem in large networks than the textbook non-convexity warning suggests; plateaus, ill-conditioning and bad initialisation are the usual culprits when training stalls.

Start with AdamW (Transformers) or SGD with momentum 0.9 (CNNs), a warm-up plus cosine schedule, gradient clipping at norm 1; then tune the peak learning rate first, because it matters more than anything else on this page.

§4.7 A few problems to think about

Problem 4A. A training set has 60,000 examples and the batch size is 256 (last partial batch kept). How many optimiser steps are taken in 10 epochs?

[hint](#) · [solution](#)



Problem 4B. Compute Adam's first step for a parameter with gradient $g_1 = -4$, $\eta = 0.01$, default betas, ε negligible. What if the gradient were -0.004 ?

[hint](#) · [solution](#)



Problem 4C. Show that for gradients with constant mean $\mathbf{g}$, $\mathbb{E}\mathbf{v}_t = (1 - \beta_2^t)\mathbb{E}\mathbf{g}^2$ for Adam's second moment with $\mathbf{v}_0 = \mathbf{0}$, and compute the correction factor after 10 steps with $\beta_2 = 0.999$.

[hint](#) · [solution](#)



Problem 4D. For $f(x) = \frac{\lambda}{2}x^2$ and gradient descent with step η , find all η for which the iterates converge, and the η that converges in one step. What happens at $\eta = 2/\lambda$?

[hint](#) · [solution](#)



Problem 4E. A gradient $(6, -8, 0)$ is clipped (i) by norm with threshold 5, (ii) by value with threshold 5. Give both results.

[hint](#) · [solution](#)



Problem 4F. With PyTorch-style momentum $\mathbf{v} \leftarrow \beta\mathbf{v} + \mathbf{g}$ and a constant gradient $\mathbf{g}$, find the limiting step and the number of steps until the step reaches 90% of it for $\beta = 0.9$ and $\beta = 0.99$.

[hint](#) · [solution](#)



5 Initialisation, normalisation and regularisation

§5.1 Initialisation, normalisation and regularisation

[STUB: Initialisation, normalisation and regularisation]

II

Architectures

Part II: Contents

6	Convolutional networks	49
6.1	Convolutional networks	49
7	Recurrent networks	51
7.1	Recurrent networks	51
8	Attention and the Transformer	53
8.1	Attention and the Transformer	53

6 Convolutional networks

§6.1 Convolutional networks

[STUB: Convolutional networks]

7 Recurrent networks

§7.1 Recurrent networks

[STUB: Recurrent networks]

8 Attention and the Transformer

§8.1 Attention and the Transformer

[STUB: Attention and the Transformer]

III

Modern Deep Learning

Part III: Contents

9	Language models and LLMs	57
9.1	Language models and LLMs	57
10	Generative and representation learning	59
10.1	Generative and representation learning	59

9 Language models and LLMs

§9.1 Language models and LLMs

[STUB: Language models and LLMs]

10 Generative and representation learning

§10.1 Generative and representation learning

[STUB: Generative and representation learning]

IV

Interview Mode

Part IV: Contents

11	Practice and PyTorch	63
11.1	Practice and PyTorch	63
12	Implement it	65
12.1	Implement it	65
13	Rapid-fire	67
13.1	Rapid-fire	67
14	Problems	69
14.1	Problems	69

11 Practice and PyTorch

§11.1 Practice and PyTorch

[STUB: Practice and PyTorch]

12 Implement it

§12.1 Implement it

[STUB: Implement it]

13 Rapid-fire

§13.1 Rapid-fire

[STUB: Rapid-fire]

14 Problems

§14.1 Problems

[STUB: Problems]



Backmatter

A Hints to problems

- 1A.** Each layer contributes $(\text{inputs} + 1) \times \text{outputs}$.
- 1B.** NAND is 0 only at $(1, 1)$.
- 1C.** Both R and γ are lengths in input space.
- 1D.** Where can the slope of f change?
- 1E.** Two are true.
- 1F.** The other half of the proof bounds $\|\mathbf{w}\|$.
- 2A.** Shift invariance.
- 2B.** Write $\tanh z = (1 - e^{-2z})/(1 + e^{-2z})$.
- 2C.** $\log \sigma(z) = -\log(1 + e^{-z})$ and $\log(1 - \sigma(z)) = -z - \log(1 + e^{-z})$.
- 2D.** Four of them.
- 2E.** Let S be the sum of the other exponentials; $p_y = e^{z_y}/(e^{z_y} + S)$.
- 2F.** What does a softmax force the 20 probabilities to do?
- 3A.** Max routes, multiply swaps.
- 3B.** Every gradient has the shape of the thing it differentiates.
- 3C.** $\delta^{(2)}$ gains a factor $\hat{y}(1 - \hat{y}) \leq \frac{1}{4}$.
- 3D.** Look at $\boldsymbol{\delta}^{(l)}$ and $\partial \mathcal{L} / \partial \mathbf{W}^{(l)}$ row by row.
- 3E.** Differentiate: $\partial \mathbf{h}^{(l+1)} / \partial \mathbf{h}^{(l)} = \mathbf{I} + \partial F / \partial \mathbf{h}^{(l)}$.
- 3F.** Where can the gradient be identically zero?
- 4A.** $\lceil N/B \rceil$ per epoch.
- 4B.** After bias correction $\hat{m}_1 = g_1$ and $\hat{v}_1 = g_1^2$.
- 4C.** Geometric series.
- 4D.** $x \leftarrow (1 - \eta\lambda)x$.
- 4E.** The norm is 10.
- 4F.** $\mathbf{v}_t = \mathbf{g}(1 - \beta^t)/(1 - \beta)$.

B Solutions to problems

- 1A.** $101 \cdot 50 = 5050$, $51 \cdot 50 = 2550$, $51 \cdot 3 = 153$; total 7753.
- 1B.** $\mathbf{w} = (-2, -2)$, $b = 3$: the sums are 3, 1, 1, -1 on $(0, 0)$, $(0, 1)$, $(1, 0)$, $(1, 1)$, so the output is 1, 1, 1, 0. NAND is functionally complete (every Boolean circuit can be rewritten with NAND gates only), and each gate is one neuron, so any Boolean function is a feed-forward network of step neurons; the depth is the circuit's depth.
- 1C.** R and γ both scale by c , so the bound is unchanged. The run itself is identical up to scale: by induction every weight vector is c times the original one, so $\text{sign}(\mathbf{w}^\top \mathbf{c}\mathbf{x})$ equals the original prediction and the same points are misclassified in the same order. (With a bias the constant coordinate 1 is not scaled and exact invariance is lost.)
- 1D.** Each term is linear except at $t = -b_k/w_k$ (if $w_k \neq 0$), where its slope jumps. So f is linear between consecutive elements of the set of at most m such breakpoints, giving at most $m + 1$ intervals. Contrast with the 2^L pieces of the depth- L sawtooth built from $2L$ units.
- 1E.** (c) and (e). (a) fails because the boundary $\sigma(z) = \frac{1}{2}$ is a line; (b) fails because affine maps compose to an affine map; (d) is backwards.
- 1F.** Each update adds $y_i \mathbf{x}_i^\top \mathbf{w}^* \geq \gamma > 0$. But $\mathbf{w}^\top \mathbf{w}^*$ could grow simply because $\|\mathbf{w}\|$ grows without $\mathbf{w}$ turning toward $\mathbf{w}^*$; the bound $\|\mathbf{w}_k\| \leq \sqrt{k}R$ shows the length grows only like $\sqrt{k}$ while the projection grows like k , so the cosine $\mathbf{w}_k^\top \mathbf{w}^* / \|\mathbf{w}_k\| \geq \sqrt{k} \gamma / R$, which cannot exceed 1, forces $k \leq (R/\gamma)^2$.
- 2A.** Both give the uniform distribution, so the loss is $\ln 4 \approx 1.386$ (row 2 of the table).
- 2B.** $2\sigma(2z) - 1 = \frac{2}{1+e^{-2z}} - 1 = \frac{1-e^{-2z}}{1+e^{-2z}} = \tanh z$. Differentiating, $\tanh'(z) = 4\sigma'(2z)$, so $\tanh'(0) = 4 \cdot \frac{1}{4} = 1$.
- 2C.** Using the hint, $\text{BCE} = \log(1 + e^{-z}) + (1 - y)z$. Its derivative is $-\frac{e^{-z}}{1+e^{-z}} + 1 - y = -(1 - \sigma(z)) + 1 - y = \sigma(z) - y$. For $z \geq 0$ the stable form is $z - zy + \log(1 + e^{-z})$, the same expression. For $z < 0$ write $\log(1 + e^{-z}) = -z + \log(1 + e^z)$; then $\text{BCE} = -zy + \log(1 + e^z) = 0 - zy + \log(1 + e^{-|z|})$.
- 2D.** (b), (d), (e), (g). ReLU and GELU are activations; softmax is an output map.
- 2E.** Halving $-\ln p_y$ means the new probability is $\sqrt{p_y}$. With $p = e^z/(e^z + S)$, the odds $p/(1 - p) = e^z/S$, so increasing z by Δ multiplies the odds by e^Δ : $\Delta = \ln \frac{\sqrt{p}/(1-\sqrt{p})}{p/(1-p)}$. For $p = 0.2$: $\sqrt{0.2} = 0.447$, odds 0.809 vs 0.25, so $\Delta = \ln 3.24 \approx 1.17$ (the script checks that the loss indeed halves).
- 2F.** The softmax makes the tags compete: probabilities sum to 1, so an image with four true tags can give each at most about 0.25, and thresholds become meaningless. Use 20 sigmoid outputs with a binary cross-entropy per tag (`BCEWithLogitsLoss` on 20 logits).
- 3A.** $m = \max(1, 4) = 4$, $f = -8$. $\bar{m} = z = -2$; the max routes it to the winner, so $\partial f / \partial y = -2$, $\partial f / \partial x = 0$; $\partial f / \partial z = m = 4$.

- 3B.** With $\Delta = \partial\mathcal{L}/\partial\mathbf{Z} \in \mathbb{R}^{32 \times 50}$: $\partial\mathcal{L}/\partial\mathbf{W} = \Delta^\top \mathbf{X}$, shape (50, 100); $\partial\mathcal{L}/\partial\mathbf{b} = \Delta^\top \mathbf{1}$, shape (50) (sum over the batch); $\partial\mathcal{L}/\partial\mathbf{X} = \Delta\mathbf{W}$, shape (32, 100).
- 3C.** $\delta^{(2)} = (\hat{y} - y)\hat{y}(1 - \hat{y})$. With $\hat{y} = 0.4917$ the factor is 0.2499, so $\delta^{(2)} = -0.1270$ and every gradient in the network is about four times smaller than with BCE; the updates shrink by the same factor (for a fixed η). Far from the target the factor is much smaller still.
- 3D.** By induction: if all rows of $\mathbf{W}^{(l)}$ are equal and all $b_j^{(l)}$ are equal, all units of layer l have the same z , the same h , and (because the outgoing weights from each are also equal) the same δ . The gradient row j is $\delta_j \mathbf{h}^{(l-1)\top}$, the same for every j , so the rows stay equal after the update. This is **symmetry**; random initialisation breaks it. The output units are distinguished by their targets, so their δ s differ and the *rows* of the output matrix become different. But within each row the entries for different hidden units stay equal (they are $\delta_j h_i$ with equal h_i), so every hidden unit keeps seeing the same combination of output errors and the hidden-layer symmetry persists.
- 3E.** The backward product becomes $\prod_k (\mathbf{I} + J_k)$. Expanding, it contains the identity term, so $\partial\mathcal{L}/\partial\mathbf{h}^{(l)}$ includes $\partial\mathcal{L}/\partial\mathbf{h}^{(L)}$ unchanged plus corrections: even if every J_k is small, the gradient does not shrink exponentially. Without the skip the product is $\prod_k J_k$, which does.
- 3F.** (i) All units dead (huge negative biases, or a huge first learning-rate step): check the fraction of zero activations per layer. (ii) The graph is cut: a `.detach()`, `.numpy()` round trip or `torch.no_grad()` in the forward pass, or the optimiser was given the wrong parameters: check that `p.grad` is non-zero after `loss.backward()`. (A learning rate of zero is the third, non-backprop cause.)
- 4A.** $\lceil 60000/256 \rceil = 235$ steps per epoch (234 full batches and one of 96), so 2350 steps.
- 4B.** $\Delta\theta = -\eta \hat{m}_1 / \sqrt{\hat{v}_1} = -0.01 \cdot (-4)/4 = +0.01$. For $g_1 = -0.004$ the step is the same $+0.01$: Adam's first step is $\eta \operatorname{sign}(g)$ whatever the scale (until $|g|$ approaches ε).
- 4C.** $\mathbf{v}_t = (1 - \beta_2) \sum_{s=1}^t \beta_2^{t-s} \mathbf{g}_s^2$, so $\mathbb{E}\mathbf{v}_t = (1 - \beta_2) \frac{1 - \beta_2^t}{1 - \beta_2} \mathbb{E}\mathbf{g}^2 = (1 - \beta_2^t) \mathbb{E}\mathbf{g}^2$. After 10 steps, $1 - 0.999^{10} \approx 0.00996$: without correction $\mathbf{v}$ would underestimate the second moment about 100-fold and the steps would be about $10\times$ too large.
- 4D.** Convergence iff $|1 - \eta\lambda| < 1$, i.e. $0 < \eta < 2/\lambda$; $\eta = 1/\lambda$ gives $x_1 = 0$. At $\eta = 2/\lambda$, $x \leftarrow -x$: the iterate oscillates forever between $\pm x_0$. Between $1/\lambda$ and $2/\lambda$ it converges while flipping sign every step (the zig-zag).
- 4E.** (i) (3, -4, 0); (ii) (5, -5, 0), which points in a different direction (both checked against PyTorch).
- 4F.** The step tends to $\eta\mathbf{g}/(1 - \beta)$: $10\eta\mathbf{g}$ or $100\eta\mathbf{g}$. It reaches 90% when $\beta^t \leq 0.1$: $t \geq \ln 0.1 / \ln 0.9 \approx 21.9$, so 22 steps for $\beta = 0.9$, and $t \geq 229.1$, so 230 steps for $\beta = 0.99$. Large β means a long memory and a slow start.

Bibliography

- [Cy89] GEORGE CYBENKO. Approximation by Superpositions of a Sigmoidal Function. In: *Mathematics of Control, Signals and Systems*, **2**: (1989) (cited p. 15)
- [HeGi16] DAN HENDRYCKS and KEVIN GIMPEL. Gaussian Error Linear Units (GELUs). In: *arXiv preprint arXiv:1606.08415*, (2016) (cited p. 21)
- [KiBa15] DIEDERIK P. KINGMA and JIMMY BA. “Adam: A Method for Stochastic Optimization”. In: *International Conference on Learning Representations (ICLR)*. 2015 (cited p. 39)
- [Li+17] TSUNG-YI LIN, PRIYA GOYAL, ROSS GIRSHICK, KAIMING HE, and PIOTR DOLLÁR. “Focal Loss for Dense Object Detection”. In: *IEEE International Conference on Computer Vision (ICCV)*. 2017 (cited p. 25)
- [LoHu19] ILYA LOSHCHILOV and FRANK HUTTER. “Decoupled Weight Decay Regularization”. In: *International Conference on Learning Representations (ICLR)*. 2019 (cited p. 41)
- [MiPa69] MARVIN MINSKY and SEYMOUR PAPERT. *Perceptrons*. MIT Press, 1969 (cited p. 13)
- [Ro58] FRANK ROSENBLATT. The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain. In: *Psychological Review*, **65**:6 (1958) (cited p. 10)
- [RuHiWi86] DAVID E. RUMELHART, GEOFFREY E. HINTON, and RONALD J. WILLIAMS. Learning Representations by Back-Propagating Errors. In: *Nature*, **323**: (1986) (cited p. 27)